

Felipe dos Santos

Análise de métodos de atualização FOTA em sistemas embarcados

Brasil

Agosto de 2025

-

Felipe dos Santos

Análise de métodos de atualização FOTA em sistemas embarcados

Monografia referente ao Trabalho Final de Graduação do curso de Engenharia de Computação da Universidade Federal de Itajubá

Universidade Federal de Itajubá – UNIFEI
Instituto de Engenharia de Sistemas e Tecnologia da Informação
Engenharia de Computação

Orientador: Otávio de Souza Martins Gomes

Brasil
Agosto de 2025

Resumo

Esta pesquisa aborda os principais desafios e requisitos técnicos para a implementação segura de atualizações remotas de firmware em sistemas embarcados. Considerando o crescimento da Internet das Coisas (IoT) e a necessidade de manter dispositivos atualizados e protegidos contra vulnerabilidades, o estudo analisa práticas recomendadas e mecanismos de segurança aplicáveis ao processo de atualização. Dentre os principais requisitos identificados, destacam-se: o uso de bootloader em dois estágios, a transmissão segura via HTTPS, a adoção de partições duplicadas para permitir rollback, e a assinatura de código. Além disso, ressalta-se a importância da adoção de medidas complementares. Como contribuição prática, a pesquisa oferece um guia replicável que pode ser utilizado por desenvolvedores e estudantes para explorar a tecnologia OTA de forma segura, promovendo sua disseminação e aplicação em diferentes contextos.

Palavras-chave: IoT. OTA. sistemas embarcados. segurança. firmware. ESP32.

Abstract

This research addresses the main challenges and technical requirements for the secure implementation of remote firmware updates in embedded systems. Considering the growth of the Internet of Things (IoT) and the need to keep devices updated and protected against vulnerabilities, the study analyzes recommended practices and security mechanisms applicable to the update process. Among the key identified requirements are the use of a two-stage bootloader, secure transmission via HTTPS, the adoption of dual partitions to enable rollback, and code signing. Additionally, the importance of complementary protection measures is highlighted. As a practical contribution, the research provides a replicable guide that can be used by developers and students to explore OTA technology safely, promoting its dissemination and application in various contexts.

Keywords: IoT. OTA. embedded systems. security, firmware. ESP32.

Lista de ilustrações

Figura 1 – Número de recalls relacionados a software nos EUA (2011-2016). Fonte: (HAMADA, 2020).	16
Figura 2 – O mercado de IoT empresarial por tecnologia (2023-2030). Adaptado de: (ANALYTICS, 2024).	17
Figura 3 – Fluxo de execução. Fonte: Elaborado pelo autor.	24
Figura 4 – Diagrama de estados FreeRTOS. Adaptado de: (FREERTOS, 2025b)	36
Figura 5 – Diagrama das entidades envolvidas na implementação. Fonte: Elaborado pelo autor.	41
Figura 6 – Exemplo da interface do menu de configurações. Fonte: Elaborado pelo autor.	42
Figura 7 – Tabela de partições padrão ESP32. Fonte: (ESPRESSIF, 2023).	43
Figura 8 – Tabela de partições para duas definições OTA do ESP32. Fonte: (ESPRESSIF, 2023).	43
Figura 9 – Organização da memória flash com duas definições OTA. Fonte: Elaborado pelo autor.	45
Figura 10 – Estrutura do projeto de implementação. Fonte: Elaborado pelo autor.	46
Figura 11 – Organização da memória flash com assinatura de código. Fonte: Elaborado pelo autor.	55
Figura 12 – Atualização OTA sem assinatura de código. Fonte: Elaborado pelo autor. . .	56
Figura 13 – Tarefa ota_update com periodicidade de 5 segundos. Fonte: Elaborado pelo autor.	58
Figura 14 – Tarefa ota_update com periodicidade de 60 segundos. Fonte: Elaborado pelo autor.	59
Figura 15 – Tríade de cibersegurança e ameaças. Fonte: Elaborado pelo autor.	62
Figura 16 – Fluxo de execução FOTA. Fonte: Elaborado pelo autor.	64
Figura 17 – Abordagens de bootloader. Fonte: Elaborado pelo autor.	69

Lista de tabelas

Tabela 1 – Lista de materiais utilizados na implementação. Fonte: Dados da pesquisa. .	41
Tabela 2 – Instruções não documentadas do ESP32. Fonte: (TARLOGIC, 2025).	66

Lista de abreviaturas e siglas

API	Application Programming Interface
CLI	Command Line Interface
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CVE	Common Vulnerabilities and Exposures
ESP32	Chip da Espressif baseado em microcontrolador com Wi-Fi/Bluetooth
ESP-IDF	Espressif IoT Development Framework
FOTA	Firmware Over-The-Air
GNU	GNU's Not Unix
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDF	IoT Development Framework (no contexto: ESP-IDF)
IP	Internet Protocol
IoT	Internet of Things
LED	Light Emitting Diode
LoRa	Long Range
LoRaWAN	Long Range Wide Area Network
LPWAN	Low-Power Wide-Area Network
MCU	Microcontroller Unit
MITM	Man-In-The-Middle
NIST	National Institute of Standards and Technology
NVS	Non-Volatile Storage
OTA	Over-The-Air

OTAU	Over-The-Air Update
RAM	Random Access Memory
ROM	Read-Only Memory
RTOS	Real-Time Operating System
SDK	Software Development Kit
SPI	Serial Peripheral Interface
SSBL	Secondary Stage Boot Loader
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver/Transmitter
WSN	Wireless Sensor Network

Sumário

1	INTRODUÇÃO	13
1.1	Objetivo	14
1.2	Justificativa	15
1.3	Estrutura do Documento	17
2	REVISÃO BIBLIOGRÁFICA	19
2.1	Computadores Embarcados	19
2.2	Arquitetura de Computadores Embarcados	20
2.3	IoT	22
2.4	Funcionamento e Arquitetura do FOTA	22
2.5	Vantagens, desvantagens e Desafios do FOTA	25
2.6	ESP32	28
2.7	O que é FOTA (Firmware Over-the-Air)	29
2.8	Infraestrutura de rede para IoT	33
2.8.1	LoRa/LoRaWAN	33
2.8.2	TCP/IP	34
2.9	FreeRTOS	35
2.9.1	Tasks	35
2.9.2	Event Groups	36
3	METODOLOGIA	39
3.1	Cronograma de atividades do TFG	39
3.2	Configuração do ambiente	40
3.2.1	Introdução	40
3.2.2	Características do ambiente e dispositivo de teste	41
3.2.3	Detalhes de implementação	41
3.3	Implementações	44
3.3.1	Código Fonte Base	45
3.3.2	Variação com Assinatura de Código	53
3.3.3	Recursos para um Cenário de Produção	56
3.4	Abordagens e Desempenho	57
3.4.1	Desempenho	57
3.4.2	Abordagem Automatizada	59
3.4.3	Abordagem Sob Demanda	60
4	ANÁLISES E RESULTADOS	61

4.1	Análise de Desempenho	61
4.1.1	Tempo de CPU	61
4.1.2	Memória Flash	61
4.2	Análise de Segurança	62
4.2.1	Implementação Utilizando HTTP/HTTPS	63
4.2.2	Implementação Utilizando Assinatura de Código	63
4.2.3	Rollback	64
4.2.4	Vulnerabilidades	65
5	CONCLUSÃO	69
	REFERÊNCIAS	71
	APÊNDICES	75
	APÊNDICE A – INFORMAÇÕES ADICIONAIS	77

1 Introdução

O avanço dos computadores digitais tem impulsionado significativamente a computação embarcada, proporcionando maior capacidade de processamento e redução de custos. Esse cenário favorece o desenvolvimento de aplicações cada vez mais sofisticadas, expandindo o potencial de diversas áreas da indústria em geral. A integração com tecnologias habilitadoras, como Inteligência Artificial, Cloud Computing, Big Data e Internet das Coisas (IoT), tem possibilitado soluções inovadoras voltadas à geração, processamento e transmissão de grandes volumes de dados (NEUSTEIN et al., 2023).

Nesse contexto, dispositivos anteriormente restritos ao processamento local como ECUs automotivas, equipamentos residenciais e embarcados em geral, passaram a desempenhar funções ampliadas, possibilitando o envio e recepção de dados por meio de conexões remotas (TINDELL, 2023).

O firmware pré-carregado em um dispositivo pode, ocasionalmente, necessitar de atualizações por diversos motivos, incluindo correções de erros pós-implantação, aprimoramentos de segurança, introdução de novas funcionalidades e melhorias de desempenho. Nesse contexto, a Atualização de Firmware Over-The-Air tem sido considerada a abordagem mais adequada, especialmente em cenários de implantação em larga escala de WSN/LPWAN, onde os nós são fisicamente inacessíveis e os métodos manuais seriam excessivamente onerosos e demorados para serem executados (PULE; ABU-MAHFOUZ, 2019).

A tecnologia FOTA atualmente está presente em diversos dispositivos. Como exemplo, pode-se citar os smartphones, que recebem atualizações dos fabricantes sem a necessidade do usuário levar o dispositivo até um determinado local. Apesar da facilidade e dos benefícios que a tecnologia apresenta, existem desafios para impulsionar essa tecnologia para outros tipos de dispositivos e aplicações. (SEN, 2010) categorizou as principais restrições de dispositivos IoT, sendo: restrições de energia, limitações de memória, comunicação não confiável, alta latência de comunicação e proteção física do dispositivo.

O presente estudo tem como objetivo sintetizar as informações disponíveis sobre a tecnologia Firmware Over-the-Air (FOTA) a partir de diversas publicações e discussões acadêmicas, a fim de estruturar uma pesquisa que aborde direcionamentos estratégicos e a identificação de riscos sob a perspectiva da segurança e da otimização de recursos na implementação dessa tecnologia. Além disso, busca-se elaborar um material técnico nacional que estabeleça requisitos mínimos para uma implementação de FOTA segura e eficiente, possibilitando sua replicação por meio de um guia prático. Dessa forma, pretende-se contribuir para o desenvolvimento de diretrizes que fomentem a adoção direcionada dessa tecnologia em diferentes contextos e aplicações.

Considerando as características que favorecem a implementação da tecnologia FOTA, este estudo utilizará um dispositivo de desenvolvimento ESP32 para realizar as implementações, abordando distintas perspectivas relacionadas à segurança e à otimização do uso de recursos. A configuração do ambiente de desenvolvimento e a implementação serão devidamente documentadas, com o intuito de elaborar um guia passo a passo que facilite a introdução ao conceito e à aplicação prática da tecnologia FOTA. Dessa forma, busca-se oferecer uma abordagem estruturada que viabilize a replicação e ampliação do conhecimento sobre essa solução em diferentes contextos tecnológicos.

1.1 Objetivo

Este trabalho tem como objetivo principal a implementação de métodos de atualização FOTA, explorando os diversos aspectos que envolvem essa tecnologia. Para isso, serão realizadas análises criteriosas quanto ao desempenho, segurança e utilização de recursos, utilizando como plataforma de desenvolvimento o microcontrolador ESP32.

Além da implementação e análise dos métodos, busca-se fornecer uma documentação técnica detalhada, facilitando a adoção da tecnologia em projetos futuros. Ao estruturar um referencial técnico robusto, pretende-se contribuir para a disseminação do conhecimento sobre atualizações FOTA em microcontroladores, ampliando suas possibilidades de aplicação prática.

Ao longo do desenvolvimento, pretende-se responder questões essenciais para a evolução da tecnologia FOTA:

- **Quais são os principais riscos de segurança associados ao FOTA e como mitigá-los?**

A atualização remota de firmware pode ser alvo de ataques, como falsificação de firmware e envenenamento do firmware. Avaliar estratégias para mitigar esses riscos é essencial para garantir um sistema confiável.

- **Como o método de atualização impacta o desempenho e eficiência do sistema?**

A forma como o dispositivo verifica e aplica novas versões de firmware influencia no consumo de recursos do sistema.

- **Qual a melhor abordagem para validar a integridade e autenticidade do firmware atualizado?**

O uso de técnicas como assinatura de código, criptografia e armazenamento de chave por diferentes técnicas pode garantir que apenas firmware legítimo seja executado no dispositivo.

- **Como a utilização de FOTA afeta a memória de armazenamento do dispositivo?**

A configuração da tabela de partições afeta o espaço disponível na memória flash e define as possibilidades de rollback, influenciando diretamente a viabilidade da atualização remota.

- **É possível oferecer um direcionamento geral para futuras implementações da tecnologia?**

Garantir a replicação dos resultados e incentivar o crescimento do tema na comunidade científica e tecnológica é essencial para a pesquisa.

Por fim, espera-se que os métodos abordados neste estudo sirvam como um ponto de partida para a aplicação da tecnologia, tornando o conteúdo acessível e bem fundamentado para desenvolvedores e pesquisadores interessados na implementação de FOTA em seus projetos.

1.2 Justificativa

O avanço dos computadores digitais tem impulsionado significativamente a computação embarcada, proporcionando maior capacidade de processamento e redução de custos. Esse cenário favorece o desenvolvimento de aplicações cada vez mais sofisticadas, expandindo o potencial de diversas áreas industriais. A integração com tecnologias habilitadoras, como Inteligência Artificial, Cloud Computing, Big Data e Internet das Coisas (IoT), tem possibilitado soluções inovadoras voltadas à geração, processamento e transmissão de grandes volumes de dados (NEUSTEIN et al., 2023).

Nesse contexto, dispositivos anteriormente restritos ao processamento local como ECUs automotivas, equipamentos residências e embarcados em geral, passaram a desempenhar funções ampliadas, possibilitando o envio e recepção de dados.

De acordo com (SYSTEMS, 2020), os dispositivos da categoria M2M, pertencentes ao ecossistema da Internet das Coisas (IoT), apresentam a maior taxa de crescimento entre os dispositivos conectados. Esse avanço é impulsionado pela crescente adoção de tecnologias emergentes que favorecem a interconectividade autônoma entre máquinas, promovendo eficiência e inovação em diversos setores da economia.

A demanda por métodos eficientes e simplificados de atualização de software em dispositivos eletrônicos tornou-se cada vez mais essencial. Os sistemas de atualização OTA surgiram como uma solução revolucionária para essa necessidade, possibilitando a distribuição remota e sem fio de atualizações e correções de software para uma ampla variedade de dispositivos eletrônicos, que vão desde smartphones e dispositivos IoT, até equipamentos médicos críticos, como ventiladores pulmonares (MEHAR et al.,).

Na área dos veículos automotivos, (HAMADA, 2020) constatou que, atualmente, um veículo automotivo possui, em média, 100 Unidades de Controle Eletrônico (ECU) e cerca de

100 milhões de linhas de código de software. Com um custo médio de \$10 por linha, esses sistemas eletrônicos são altamente onerosos, e os recalls resultam em perdas bilionárias, além de prejudicarem a reputação dos fabricantes. Essa situação tende a se agravar, pois o número de ECUs está crescendo rapidamente, e especialistas preveem que, em algum momento de 2020, um carro poderá possuir até 300 ECUs. Diante desse cenário, as montadoras (OEMs) precisam minimizar o impacto dos recalls por meio de uma gestão eficiente do software ao longo do ciclo de vida do veículo. A [Figura 1](#) mostra os dados de recall por software dos veículos nos EUA para o período informado.

Existem diversas restrições para a atualização do software dos automóveis:

1. **Tempo:** a atualização do firmware pode levar vários dias.
2. **Complexidade:** o processo de atualização deve ser totalmente gerenciado na camada de aplicação.
3. **Custo:** para atualizar o firmware do veículo, os proprietários precisam se deslocar até os centros de manutenção.

A necessidade de otimizar esse processo impulsiona o desenvolvimento de soluções mais eficientes para gerenciamento e atualização remota dos sistemas embarcados.

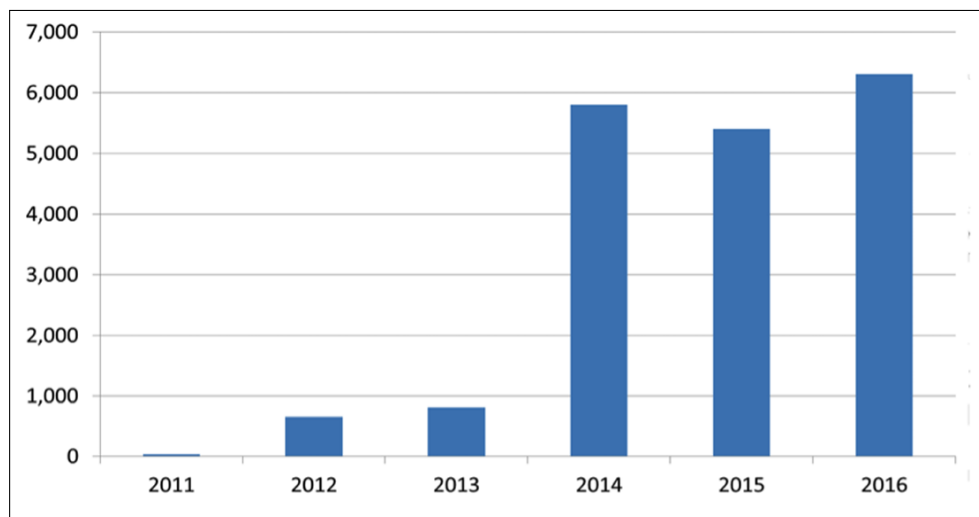


Figura 1 – Número de recalls relacionados a software nos EUA (2011-2016). Fonte: ([HAMADA, 2020](#)).

De acordo com ([ANALYTICS, 2024](#)), o investimento na área de Internet das Coisas (IoT) tem demonstrado um crescimento contínuo por parte da indústria, impulsionado pela expansão da conectividade e pela necessidade de inovação tecnológica. Conforme apresentado na [Figura 2](#), observa-se a distribuição anual do investimento em IoT entre as categorias hardware, software, serviços e segurança, evidenciando a diversificação dos aportes financeiros no setor.

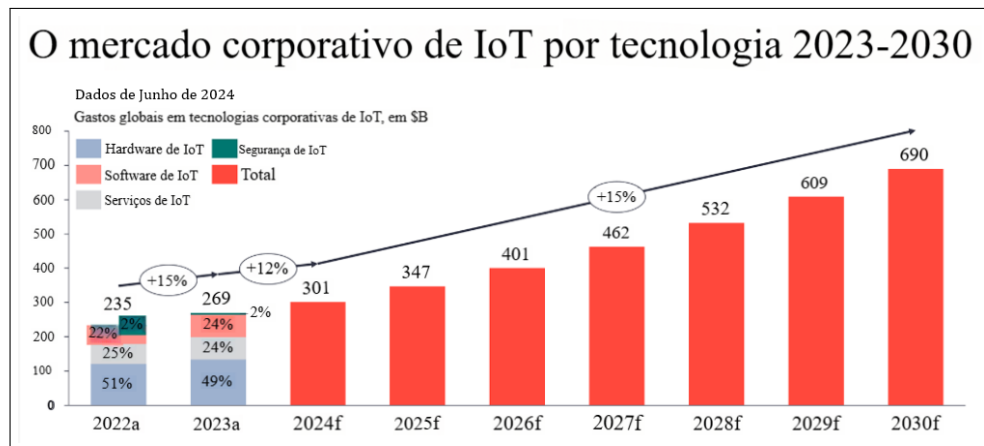


Figura 2 – O mercado de IoT empresarial por tecnologia (2023-2030). Adaptado de: (ANALYTICS, 2024).

Com o aumento da complexidade das aplicações, os avanços constantes na tecnologia e o crescimento do número de dispositivos conectados, torna-se essencial que esses dispositivos disponham de mecanismos de atualização que viabilizem modificações remotas e dinâmicas, sem a necessidade de acesso físico ao equipamento. A atualização FOTA permite a adição de novas funcionalidades, a correção de falhas e a melhoria da qualidade de serviço (QoS) em dispositivos embarcados e IoT em larga escala, garantindo maior eficiência e longevidade operacional. A abordagem prática do FOTA é essencial para aprofundar a compreensão do tema e estabelecer diretrizes para uma implementação segura e eficiente (PULE; ABU-MAHFOUZ, 2019).

1.3 Estrutura do Documento

O presente estudo consiste em uma monografia desenvolvida como requisito para a conclusão do curso de Engenharia de Computação. Com base nos objetivos e na justificativa apresentados, a pesquisa será estruturada da seguinte maneira para aprofundar a discussão proposta:

1. Referencial Teórico – Inicialmente, serão apresentados conceitos fundamentais e informações de especialistas sobre a tecnologia FOTA e tópicos relacionados. Serão exploradas boas práticas, desafios técnicos e requisitos essenciais, fornecendo uma base teórica sólida para a implementação.
2. Metodologia – A implementação será realizada considerando diferentes configurações de segurança, incluindo o uso de HTTP e HTTPS, além da implementação de assinatura de código. Cada uma dessas abordagens será testada quanto ao impacto na segurança, desempenho e consumo de recursos do ESP32.
3. Análise e Resultados – Os dados coletados serão organizados em gráficos e tabelas para permitir uma comparação detalhada entre as diferentes configurações testadas. Essa análise

ajudará a identificar vantagens, limitações e recomendações para futuras implementações da tecnologia FOTA.

2 Revisão Bibliográfica

2.1 Computadores Embarcados

Os computadores embarcados são sistemas computacionais integrados a dispositivos que não são comercializados como computadores tradicionais. Também conhecidos como microcontroladores, esses sistemas desempenham um papel fundamental na gestão dos dispositivos, controlando suas funcionalidades e a interface do usuário. Os microcontroladores estão presentes em uma ampla gama de equipamentos, incluindo dispositivos médicos, brinquedos, eletrodomésticos e veículos, entre outros.

Além disso, os sistemas embarcados frequentemente operam sob restrições físicas significativas, como limitações de tamanho, peso e consumo de energia, além de exigências específicas relacionadas a fatores elétricos e mecânicos (TANENBAUM, 2016). Essas restrições demandam arquiteturas otimizadas e soluções eficientes para garantir desempenho, confiabilidade e autonomia no funcionamento desses dispositivos.

Os sistemas embarcados atualmente abrangem desde implementações simples e dedicadas até soluções tecnologicamente avançadas, tanto em termos de funcionalidade quanto de capacidade computacional. O software desses sistemas pode ser executado em sistemas operacionais em tempo real (RTOS), garantindo eficiência e previsibilidade na execução de tarefas críticas.

A comunicação entre dispositivos embarcados é viabilizada por meio de conexões guiadas tradicionais, como RS485, CAN e Ethernet, que oferecem estabilidade e confiabilidade em ambientes industriais e automotivos. Paralelamente, o avanço das tecnologias sem fio possibilitou a ampla adoção de protocolos como Bluetooth, WiFi e GSM/GPRS, permitindo maior flexibilidade na interconectividade e viabilizando aplicações distribuídas e remotas (BILEK; RUZICKA, 2003).

Os processadores embarcados existem há muito tempo, tendo sido inicialmente desenvolvidos para aplicações laboratoriais e industriais. Durante anos, seu principal uso foi o controle industrial, com dispositivos de baixo custo e desempenho limitado. Um exemplo disso é o microcontrolador de 8 bits, ainda amplamente utilizado em automóveis, eletrodomésticos e dispositivos simples.

A partir do final dos anos 1980 e início dos anos 1990, novas oportunidades surgiram para esses processadores, impulsionando avanços em videogames, set-top boxes, celulares e assistentes digitais pessoais. O crescimento da conectividade e dos dispositivos inteligentes aumentou significativamente a demanda por processadores embarcados, elevando também os requisitos de desempenho para atender às novas aplicações tecnológicas (HENNESSY; PATTERSON, 2017).

2.2 Arquitetura de Computadores Embarcados

Um microcontrolador (MCU) é um pequeno computador em um único circuito integrado, composto por uma unidade central de processamento (CPU) simples, combinada com dispositivos periféricos, como memórias, dispositivos de entrada e saída (I/O) e temporizadores. Segundo algumas estimativas, mais da metade de todos os CPUs vendidos mundialmente são microcontroladores, embora essa afirmação seja difícil de comprovar, devido à indefinição na distinção entre microcontroladores e processadores de uso geral (LEE; SESHIA, 2016).

A CPU executa cada instrução por meio de uma série de pequenos passos. De forma simplificada, o processo segue a seguinte sequência:

1. Buscar a próxima instrução da memória e carregá-la no registrador de instrução.
2. Alterar o contador de programa para apontar para a próxima instrução.
3. Determinar o tipo da instrução recém-carregada.
4. Se a instrução utilizar uma palavra da memória, identificar sua localização.
5. Buscar a palavra na memória, se necessário, e armazená-la em um registrador da CPU.
6. Executar a instrução.
7. Retornar ao passo 1 para iniciar a execução da próxima instrução.

Essa sequência de etapas é comumente denominada ciclo buscar-decodificar-executar e constitui o núcleo operacional de todos os computadores (TANENBAUM, 2016).

A memória é a parte do computador onde os programas e dados são armazenados. Sem uma memória da qual os processadores possam ler e escrever informações, não existiriam computadores digitais com programas armazenados (TANENBAUM, 2016).

Muitas aplicações exigem grandes quantidades de memória, superiores à capacidade disponível on-chip em um microcomputador. Para lidar com essa limitação, muitos processadores utilizam uma hierarquia de memória, combinando diferentes tecnologias para aumentar a capacidade total, ao mesmo tempo em que otimizam custos, latência e consumo de energia. O programador pode não perceber que a memória está fragmentada entre essas tecnologias. Para tornar esse processo transparente, um esquema amplamente utilizado chamado memória virtual permite que essas diferentes tecnologias sejam apresentadas ao programador como um espaço de endereçamento contínuo, simplificando o gerenciamento de recursos pelo sistema (LEE; SESHIA, 2016).

Vale destacar que, ao lidar com memórias em programação, principalmente em baixo nível, existem algumas divergências nas unidades de medida entre bits e bytes. (TANENBAUM,

2016) destaca que na computação, unidades de medição de memória, disco, arquivos e bancos de dados seguem convenções distintas em relação ao sistema decimal. Na prática da indústria, o termo "quilo" refere-se a $2^{10}=1024$, em vez de $10^3=1000$, pois a capacidade da memória é sempre baseada em potências de dois. Isso cria diferenças entre unidades binárias e decimais, gerando discrepâncias entre os valores anunciados pelos fabricantes e os reconhecidos pelo sistema operacional.

As memórias voláteis precisam de energia elétrica para manter os dados armazenados e perdem as informações quando o sistema é desligado. SRAM (Static RAM): Oferece acesso rápido aos dados e é utilizada na memória cache dos processadores. Sua estrutura baseada em transistores evita a necessidade de atualizações constantes. DRAM (Dynamic RAM): Utilizada como memória principal dos computadores, depende da recarga periódica dos capacitores para preservar as informações, tornando-a mais lenta que a SRAM, mas mais acessível em termos de custo e capacidade.

As memórias não voláteis mantêm os dados armazenados mesmo sem energia elétrica, sendo ideais para armazenamento permanente. ROM (Read-Only Memory): A ROM armazena dados de forma fixa e não pode ser alterada ou apagada facilmente. É usada para armazenar firmware e instruções essenciais para o funcionamento de dispositivos, como o BIOS dos computadores. Algumas variações incluem PROM, EPROM e EEPROM, que permitem modificações limitadas. Memória Flash: Derivada da EEPROM, a memória Flash é amplamente usada em pen drives, SSDs e cartões de memória, permitindo leitura e escrita rápidas com alta durabilidade.

Um mapeamento de memória para um processador define como os endereços são associados ao hardware. O tamanho total do espaço de endereçamento é limitado pela largura do endereço do processador. Por exemplo, um processador de 32 bits pode endereçar 2^{32} locais de memória, o que equivale a 4 gibibytes (GiB), assumindo que cada endereço se refere a um único byte.

A arquitetura Harvard é um modelo computacional que separa a memória de programa da memória de dados, garantindo que instruções e dados sejam armazenados em unidades de memória distintas e acessados por barramentos separados. Essa separação permite que o processador busque instruções e acesse dados simultaneamente, aumentando a eficiência e o desempenho, especialmente em sistemas embarcados e aplicações de processamento de sinal (PAKPAHAN et al.,).

Por outro lado, a arquitetura de von Neumann utiliza um único espaço de memória para armazenar tanto instruções quanto dados, fazendo com que o processador os acesse de forma sequencial pelo mesmo barramento. Embora essa abordagem simplifique o design do hardware, ela pode gerar um gargalo de desempenho, conhecido como gargalo de von Neumann, onde a velocidade de processamento é limitada pela taxa de busca e transferência de instruções e dados (PAKPAHAN et al.,).

Uma interrupção é um mecanismo para pausar a execução do que quer que um processador esteja realizando no momento e executar uma sequência de código predefinida chamada rotina de serviço de interrupção (ISR) ou manipulador de interrupção (LEE; SESHIA, 2016).

Para gerar uma base de tempo confiável os microcontroladores possuem um circuito dedicado para realizar contagem. Quando este circuito é alimentado por um sistema temporizado, ou clock, a contagem obedece a uma constante de tempo. A junção do circuito de contagem com o sistema de clock é chamada de temporizador (ALMEIDA; MORAES; SERAPHIM, 2017).

2.3 IoT

IoT, um conceito que abriga uma rede de objetos inteligentes ubíquos, tornou-se parte essencial da vida cotidiana moderna, viabilizando novas aplicações e serviços que vão desde a automação residencial até dispositivos médicos embutidos e equipamentos pessoais. A conectividade rápida e sensores inteligentes permitem que esses dispositivos IoT colem, processem e transmitam informações de maneira eficiente e integrada. Tecnologias avançadas, leves e de baixo consumo de energia tornaram ainda mais viável o uso de dispositivos IoT em locais remotos, exigindo pouca ou nenhuma observação física e manutenção (RAHMAN et al., 2017).

Para (HAMADA, 2020), a Internet das Coisas abrange um conjunto amplo de tecnologias e casos de uso, tornando difícil estabelecer uma definição única e precisa que englobe tudo. De fato, o conceito pode ser estendido para qualquer dispositivo que seja capaz de receber, processar e transmitir informações. Dispositivos embarcados estão cada vez mais entrando no conceito de IoT. Por exemplo, um eletrodoméstico com um computador digital dedicado ao controle do funcionamento, hoje é capaz de receber e enviar informações de outros dispositivos, assim como alterar o funcionamento com base nas informações processadas.

2.4 Funcionamento e Arquitetura do FOTA

Firmware Over-The-Air (FOTA) é uma tecnologia de gerenciamento de software de dispositivo móvel na qual o firmware operacional de um dispositivo móvel é atualizado remotamente pelo fabricante, sem a necessidade de intervenção manual do usuário. Telefones compatíveis com FOTA baixam diretamente as atualizações do provedor de serviços, garantindo melhorias contínuas de desempenho, segurança e novos recursos sem precisar de conexão física a um computador (HAMADA, 2020).

A implementação da tecnologia FOTA varia conforme a aplicação, exigindo trade-offs quando o dispositivo possui recursos limitados. No entanto, certos parâmetros são comuns às implementações existentes e essenciais para futuras aplicações, garantindo compatibilidade, segurança e eficiência no processo de atualização.

Um componente chave para a inicialização de um dispositivo digital que executa um firmware ou um sistema operacional é o bootloader. O bootloader é um pequeno bloco de software que viabiliza a inicialização de um sistema operacional ou firmware e permite a execução de atualizações. Além de preparar o ambiente de hardware para a execução do sistema, ele pode incluir mecanismos de segurança para garantir a integridade do processo, validar assinaturas digitais e oferecer suporte a atualizações remotas, como as realizadas via FOTA. Em sistemas embarcados, desempenha um papel fundamental na comunicação entre os componentes de hardware e software, assegurando uma transição eficiente entre o estado de inicialização e o funcionamento pleno do dispositivo (BIELAWSKI et al., 2020)

A implementação de um bootloader de dois estágios no contexto de atualizações FOTA para dispositivos móveis é amplamente debatida como um requisito essencial para garantir segurança e qualidade de serviço. Essa abordagem permite maior controle sobre o processo de atualização, reduzindo riscos de falhas e protegendo o dispositivo contra possíveis vulnerabilidades durante a transferência e instalação do novo firmware.

(BROWN, 2018) apresenta o conceito como um trade-off necessário. O bootloader primário é um aplicativo de software que permanece permanentemente na memória somente leitura do microcontrolador. A região onde ele está armazenado é conhecida como espaço de informações e, em alguns casos, não é acessível aos usuários. Esse aplicativo é executado sempre que ocorre um reset, geralmente realizando inicializações essenciais do hardware e podendo carregar o software do usuário na memória. No entanto, se o microcontrolador possuir memória não volátil integrada, como a flash, o bootloader pode simplesmente transferir o controle para o programa armazenado na flash, sem necessidade de carregamento. Caso o bootloader primário não ofereça suporte a atualizações OTA, torna-se necessário um bootloader de segundo estágio. Assim como o primário, o SSBL ou bootloader secundário, será executado a cada reset, mas incluirá uma parte do processo de atualização OTA.

O bootloader primário sempre executa o programa cuja Interrupt Vector Table (IVT) está localizada no endereço 0, onde são definidos os vetores de interrupção. Para modificar o destino do reset, é necessária a realocação da IVT. Se houver uma interrupção de energia durante esse processo, o sistema pode ficar permanentemente corrompido e inoperante, comprometendo sua funcionalidade. Dessa maneira, um bootloader de segundo estágio permite a fixação do endereço 0 para o bootloader de segundo estágio, possibilitando a alteração da inicialização de firmware sem alterar o endereço inicial de reinicialização do dispositivo.

Dr. Ken Tindell, especialista na área de desenvolvimento eletrônico automotivo, descreveu uma situação que ocorreu durante o primeiro carro da Volvo a utilizar memória flash (TINDELL, 2023). O segundo estágio do bootloader foi necessário para evitar falhas catastróficas causadas por apagamento acidental da memória flash. A preocupação era que um bug no firmware pudesse apagar a si próprio, potencialmente tornando veículos inoperantes em escala global. Embora tenha sido introduzido um código de desbloqueio para evitar o apagamento

acidental da memória flash, isso apenas transferiu o risco para o software de programação da flash. O bootloader primário continha um código mínimo, responsável por se comunicar via CAN bus e escrever dados na RAM. Já o bootloader secundário, carregado na RAM, incluía o código de programação da flash e era utilizado para atualizar o firmware. Essa abordagem garantia que o código de programação da flash não estivesse presente durante a operação normal do veículo, reduzindo significativamente os riscos de apagamento acidental e permitindo recuperação segura em caso de falhas durante o processo de atualização.

A [Figura 3](#) demonstra o fluxo de inicialização de um bootloader de dois estágios. Após ligado, os componentes principais do dispositivo são inicializados pelo primeiro estágio, quem faz uma branch para o segundo estágio. O segundo estágio fica responsável por inicializar os recursos utilizados pelo desenvolvedor, como a atualização OTA, procedimentos de verificação e segurança e outros. A partir desse momento, o bootloader de segundo estágio faz uma branch para o firmware, finalizando o fluxo.

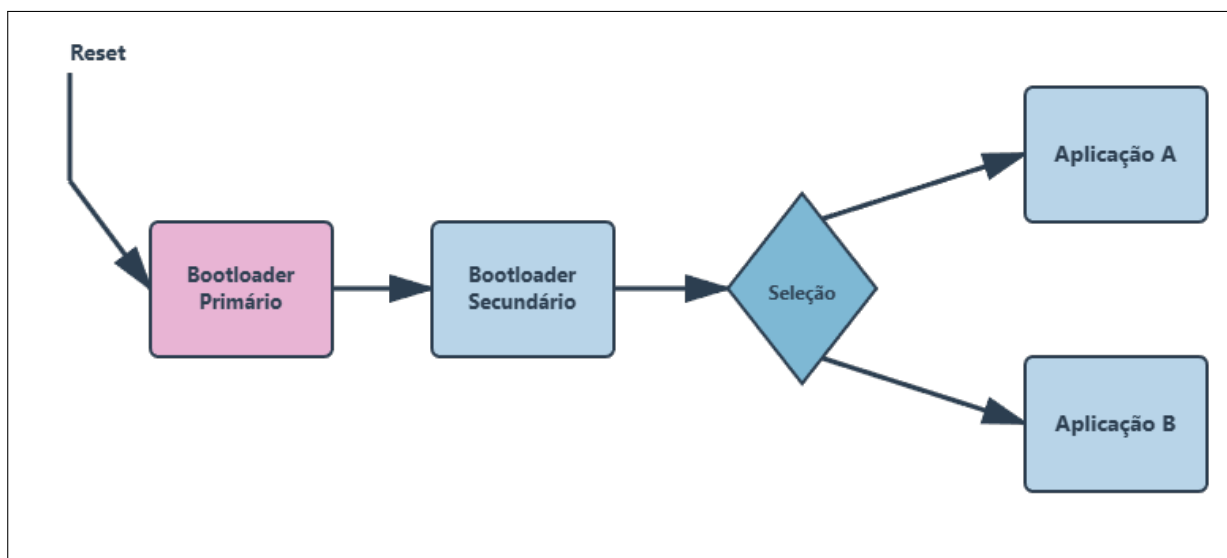


Figura 3 – Fluxo de execução. Fonte: Elaborado pelo autor.

Para o processo de atualização, (BIELAWSKI et al., 2020) apresentou uma abordagem abrangente para o processo de atualização FOTA, estruturada em cinco etapas essenciais: empacotamento, transporte, recepção, transmissão, instalação, verificação e manutenção. Cada fase desempenha um papel fundamental na garantia da integridade, segurança e eficiência do processo de atualização, assegurando que o firmware seja corretamente distribuído, aplicado e validado no dispositivo-alvo.

Os dados necessários para a atualização de uma ECU são organizados e preparados para o transporte, garantindo sua integridade até o destino, como centros de serviço, concessionárias ou proprietários de veículos. Esse processo também inclui documentação e medidas de segurança para prevenir corrupção ou adulteração durante a transmissão.

Para que a atualização chegue ao local onde será aplicada, diferentes métodos de transporte podem ser utilizados, desde a entrega por mídia física até a transmissão segura via conexão HTTPS por modem celular. Caso essa etapa não esteja devidamente protegida, há risco de ataques que podem comprometer a integridade dos pacotes, desviando-os ou substituindo-os por versões alteradas.

O pacote, ao alcançar a ECU designada, passa por uma verificação de segurança. Durante essa fase, assinaturas digitais são autenticadas e a consistência dos dados é analisada para garantir que não tenham sido corrompidos ou modificados antes da instalação.

A aplicação da atualização no dispositivo ocorre de maneira controlada e segura. Com ECUs interconectadas, uma unidade pode ser utilizada para programar outra, reduzindo a necessidade de múltiplas intervenções. No entanto, qualquer ponto de acesso externo, como conexões sem fio ou portas de serviço, deve ser protegido para evitar modificações não autorizadas.

Após a atualização, medidas são adotadas para monitorar seu impacto e assegurar que tenha sido concluída corretamente. Os fabricantes analisam o status e autenticidade das versões instaladas, identificando possíveis falhas ou alterações feitas pelos usuários. Em casos de erro, o sistema pode reverter para uma versão anterior ou enviar dados ao fabricante para investigação detalhada.

2.5 Vantagens, desvantagens e Desafios do FOTA

No geral, a tecnologia FOTA tornou-se indispensável para o monitoramento remoto de dispositivos, diagnósticos, e distribuição e atualização eficiente de firmware, incluindo respostas rápidas a vulnerabilidades de segurança ([PARK; LEE; LEE, 2025](#)). O processo de atualização via FOTA permite que uma grande quantidade de dispositivos seja atualizada simultaneamente, garantindo eficiência e praticidade. Isso pode ser feito com um único comando, enviado pelo sistema central, ou por meio da disponibilização da atualização em um repositório, onde os dispositivos conectados podem acessá-la automaticamente.

Na área de IoT, por exemplo, ([ARAKADAKIS et al., 2021](#)) mencionou oportunidades para a tecnologia FOTA. Tradicionalmente, para atualizar os nós de uma Rede de Sensores Sem Fio (WSN), que frequentemente compõem sistemas IoT, era necessário o envio de equipes de manutenção para acessá-los via porta serial ou outro canal físico. No entanto, essa abordagem não é escalável e demanda muito tempo, o que pode ser crítico quando as atualizações envolvem correções de segurança urgentes. Além disso, o acesso físico pode ser impossível em alguns casos, como quando os sensores estão embutidos em pavimentos ou implantados no corpo humano para aplicações médicas.

Independentemente da atenção dada durante o período de desenvolvimento, erros de software podem ocorrer em qualquer nível do sistema e em qualquer estágio do ciclo de desen-

volvimento. Uma combinação inesperada de entradas recebidas pelos nós de uma rede pode ativar trechos de firmware não testados, resultando em nós inoperantes que podem degradar a Qualidade de Serviço (QoS) ou até comprometer a integridade da rede. Por isso, atualizações de firmware são frequentemente lançadas para corrigir esses erros e falhas de segurança, introduzir novas funcionalidades ou até mesmo modificar completamente o propósito da aplicação.

A aviação comercial e os dispositivos médicos são indústrias adjacentes que parecem oferecer semelhantes contribuições em relação a indústria automobilística, pois ambas são aplicações críticas para a segurança e apresentam algumas semelhanças nas arquiteturas de plataforma e dispositivos (BIELAWSKI et al., 2020). O caso do recall devido a problemas de software em veículos citado anteriormente pode ser expandido para a aviação e a área médica. Dispositivos que são fabricados com algum tipo de mau funcionamento ou que necessitam da implementação de novas funcionalidades podem recebê-las remotamente, sem a necessidade de ação do usuário.

Em uma entrevista com o engenheiro Suresh Alla (STENBERG, 2021), que atua com foco em tecnologia associada a dispositivos médicos eletrônicos, apresenta alguns pontos positivos da aplicação da tecnologia na área. Em aparelhos domésticos como, por exemplo, um dispositivo para tratamento de enxaquecas por plugues no ouvido, Suresh cita que a atualização over-the-air poderia ser utilizada para melhoramentos de UI/UX ou até mesmo para atualizações de firmware para adequar ao comportamento do cliente. Para outros aparelhos médicos de baixo e alto custo, a atualização over-the-air pode diminuir o tempo do lançamento de novas funcionalidades, além de colaborar para a correção de falhas críticas. Alguns dispositivos executam modelos de inteligência artificial. Suresh explica que os modelos são treinados na nuvem, mas a inferência do modelo ocorre no dispositivo em questão, portanto, novamente, é muito importante poder lidar com atualizações de software OTA aqui. O aprendizado contínuo em aprendizado de máquina e aprendizado profundo significa que novos modelos serão implantados da nuvem e enviados para os dispositivos em campo. A cadência precisa ser frequente.

Para sintetizar os benefícios da tecnologia FOTA, pode-se definir com base na abordagem dos autores citados que a capacidade de alterar o funcionamento de um ou vários dispositivos remotamente em qualquer seguimento da indústria, apresenta grande potencial econômico. Além disso, a tecnologia possibilita grande escalabilidade para a manutenção de novos dispositivos. A Tesla é uma das empresas que se beneficiaram do FOTA, principalmente após detectar a possibilidade de otimizar a lógica de frenagem do seu Model 3. Com a atualização do sistema de freios via FOTA, a distância de frenagem foi significativamente reduzida, passando de um risco potencial de 46 metros para um mais gerenciável 40 metros, melhorando consideravelmente a segurança na condução (LI et al., 2024).

No que tange à aplicabilidade, é importante destacar que, assim como qualquer tecnologia de software, sempre há um custo para o hardware envolvido. No caso de dispositivos embarcados e IoT, esse custo é ainda mais significativo, pois esses dispositivos apresentam limitações

intrínsecas que impactam diretamente sua capacidade de processamento e armazenamento. (SEN, 2010) categorizou as principais restrições de dispositivos IoT em restrições de energia, limitações de memória, comunicação não confiável, alta latência de comunicação e proteção física do dispositivo.

Para a implementação de FOTA, de acordo com os autores citados, a tecnologia FOTA pode se tornar uma desvantagem caso alguns requisitos de implementação não sejam satisfeitos. O aumento da exposição do dispositivo para além do acesso físico, torna a gravidade de uma falha de segurança consideravelmente maior. Imagine que um caso em que um invasor consiga acesso ao ambiente que contem o firmware para a atualização de uma classe de dispositivos, nesse caso, o invasor teria acesso a todos os dispositivos da classe em questão. No caso de um veículo vendido mundialmente sendo atualizado via FOTA, por exemplo, a alteração do funcionamento do freio do dispositivo poderia ocasionar acidentes de escala global.

Uma consideração importante ao trabalhar com microcontroladores é a possível presença de software oculto e funcionalidades embutidas pelo fabricante do dispositivo (CORPORATION, 2012). Dois casos notáveis ilustram essas preocupações. O primeiro envolveu um dispositivo de monitoramento de saúde usado no setor de Saúde e Saúde Pública (HPH) dos EUA. Seu firmware continha um endereço IP codificado, criando um risco potencial de exposição de dados (CISA, 2025). O segundo caso diz respeito ao microcontrolador ESP32 usado nesta pesquisa. O fabricante incluiu 29 comandos Bluetooth HCI (Host Controller Interface) não documentados, criando efetivamente uma porta dos fundos que permite controle total sobre o dispositivo (TARLOGIC, 2025). Essa vulnerabilidade foi registrada oficialmente no banco de dados Common Vulnerabilities and Exposures (CVE) como CVE-2025-0626 (CORPORATION, 2025a). A presença de software oculto pode contribuir para a exposição do software.

Eventos que podem levar à corrupção de dados, como falha de energia durante a gravação do software na memória flash, podem tornar uma aplicação ECU inutilizável até que uma atualização bem-sucedida seja realizada. Esse risco reforça a necessidade de mecanismos de atualização seguros e à prova de falhas, garantindo a integridade e funcionalidade do sistema (BIELAWSKI et al., 2020).

A segurança para o processo de atualização FOTA é uma desvantagem considerável com relação aos métodos de acesso por meios de acesso guiado e presencial. O tampering físico, isto é, a alteração de configurações indevidas por um agente externo que tenha acesso físico ao dispositivo, era a vulnerabilidade com maior grau de risco.

(MEHAR et al.,) destaca que as principais desvantagens das atualizações OTA giram em torno de três desafios principais:

1. **Transmissão segura de dados** – Como as atualizações OTA envolvem a transferência de firmware ou software por meio de redes, garantir criptografia de ponta a ponta e proteção contra ameaças cibernéticas é fundamental.

2. **Integridade dos dados** – A atualização deve manter sua integridade, ou seja, os arquivos não podem ser corrompidos ou alterados durante a transmissão, pois isso pode causar falhas ou vulnerabilidades nos dispositivos que a recebem.
3. **Confiabilidade da infraestrutura de rede** – Uma infraestrutura de rede estável e **de alta qualidade** é necessária para evitar interrupções, atualizações incompletas ou falhas que poderiam deixar os dispositivos inoperantes.

Esses pontos são especialmente relevantes em aplicações críticas, como sistemas automotivos, dispositivos médicos e aviação, onde falhas ou brechas de segurança podem ter consequências graves.

2.6 ESP32

A programação para sistemas embarcados é intimamente ligada à arquitetura do processador, aos tipos de periféricos disponíveis no microcontrolador e ao modo com que esses periféricos foram conectados com os demais circuitos na placa. Conhecer o sistema é, portanto, fundamental para se construir um programa que funcione da maneira desejada ([ALMEIDA; MORAES; SERAPHIM, 2017](#)).

Existem duas abordagens principais para interagir com o microcontrolador ESP32, o framework Arduino Core e o framework ESP-IDF. Se escolhido o Arduino Core, o programador utilizará abstrações da linguagem proprietária do Arduino para interagir com o ESP32. Por questões de granularidade e compatibilidade, nesse trabalho serão avaliados somente métodos implementados utilizando o ESP-IDF.

O microcontrolador ESP32 possui bibliotecas proprietárias para implementação da atualização de firmware por meio do protocolo HTTPS. A biblioteca "ESP HTTPS OTA" é a mais recente e fornece APIs para realizar a atualização do dispositivo remotamente por meio do protocolo HTTP/HTTPS. O protocolo HTTPS exige que o servidor que irá conter o binário de atualização possua um certificado SSL e o mesmo deve ser informado no firmware contido no dispositivo a ser atualizado. Dessa maneira, o dispositivo receberá somente arquivos binários do servidor conhecido pelo firmware atual. Além disso, existe a opção de utilizar assinaturas digitais nos arquivos de atualização com verificação, utilizando o conceito de assinatura de código.

O microcontrolador ESP32 da Espressif possui um bootloader de dois estágios embutido. O bootloader do segundo estágio pode ser alterado ou completamente substituído e, caso ocorra algum erro durante o processo de inicialização, não é necessário regravar o bootloader por meio de portas físicas como JTAG ou SPI.

O fluxo de inicialização do dispositivo segue as seguintes etapas:

1. O bootloader de primeiro estágio na ROM carrega o bootloader de segundo estágio contido na memória flash a partir do endereço 0x1000.
2. O bootloader de segundo estágio carrega a tabela de partição e a função "main" a partir da memória flash.
3. Ocorre a inicialização da aplicação do usuário (inicialização de uma função main ou de um escalonador de um sistema operacional instalado por exemplo).

Quando o bootloader de segundo estágio é alterado, é possível que o novo bootloader exija um maior espaço na memória flash. Por padrão, a tabela de partição é armazenada a partir do endereço 0x8000. Esse desvio pode ser alterado para possibilitar maior espaço para o bootloader caso necessário. Além das partições padrões do sistema, existe a possibilidade de configurá-las e selecionar diferentes modos. Caso seja solicitada a atualização via OTA, a tabela de partição reserva outras partições para a configuração OTA e o novo software a ser armazenado no dispositivo.

Com relação a segurança, o ESP32 oferece recursos como inicialização segura e criptografia da flash. A verificação de autenticidade do firmware no processo de inicialização possibilita que apenas softwares autorizados possam ser executados pelo dispositivos, o que resulta em um processo de verificação por software de chaves públicas e chaves privadas antes da execução do conteúdo de uma partição. Alguns modos de operação desabilitam a interface JTAG se necessário. Além disso, existe a possibilidade de criptografar o conteúdo da memória flash externa, o que torna a leitura física dos dados gravados no dispositivo ineficaz ([ESPRESSIF, 2023](#)).

2.7 O que é FOTA (Firmware Over-the-Air)

Os computadores embarcados são computadores embutidos em dispositivos que não são vendidos como computadores. Os computadores embarcados, às vezes chamados de microcontroladores, gerenciam os dispositivos e controlam a interface do usuário. Os microcontroladores são encontrados em uma grande variedade de dispositivos diferentes como dispositivos médicos, brinquedos, eletrodomésticos, veículos etc. Sistemas embarcados muitas vezes têm restrições físicas em termos de tamanho, peso, consumo de bateria e outros limites elétricos e mecânicos ([TANENBAUM, 2016](#)). Assim como computadores convencionais, esses dispositivos possuem um sistema e podem necessitar de atualizações, seja para adicionar novas funcionalidades ou corrigir possíveis erros de funcionamento.

Sistemas embarcados hoje em dia contêm implementações simples e dedicadas e sistemas relativamente sofisticados (funcionalmente e computacionalmente). Seu software é executado em, por exemplo, sistemas operacionais em tempo real e a comunicação ao redor é fornecida por uma conexão guiada clássica (RS485, CAN, Ethernet) ou via conexão sem fio mais usada atualmente (Bluetooth, WiFi, GSM/GPRS) ([BILEK; RUZICKA, 2003](#)).

Recentemente, existem vários ataques distribuídos globais de negação de serviço (DDoS) contra dispositivos IoT. Em setembro de 2016, mais de 140.000 câmeras de uma empresa de hospedagem francesa OVH foi hackeada. Em novembro de 2016, mais de 900.000 roteadores da Deutsche Telekom (DT) foram infectados por um malware chamado Mirai. Mirai é um malware que transforma esses dispositivos IoT em “bots” controlados remotamente que podem ser usados como parte de uma botnet em um ataque de rede em larga escala. O exposto demonstra a alta vulnerabilidade dos dispositivos IoT e a falta de consideração dos fabricantes. Como manter o firmware atualizado é um grande problema (TENG et al., 2017).

As Redes de Sensores Sem Fio (WSN) fornecem uma promissora infra-estrutura para inúmeras aplicações que vão desde rastreamento, automação agrícola, monitoramento ambiental, telemedicina e vigilância militar. Com um simples arquitetura, uma WSN tem permitido que controle e monitoramento de processos sejam conduzidos remotamente, em tempo real, com mínima intervenção humana e a um custo relativamente baixo. O sistema típico de WSN consiste em dois subsistemas principais, ou seja, nós e gateways. Os nós são dispositivos equipados com detecção, processamento e comunicação sem fio e, juntos, podem colaborar para realizar um processo de medição. Gateways fornecem um meio para coletar dados de nós e retransmiti-los para centros de armazenamento, análise motores e sistemas de visualização de dados (PULE; ABU-MAHFOUZ, 2019).

Convencionalmente, FOTA ou OTA foi denominado Over-The-Air Programação (OTA/O-TAP), e geralmente referido como um processo de atualização do firmware do dispositivo através de um meio de comunicação sem fio. O firmware é pré-carregado nos dispositivos remotos ou na rede dos sensores para dispositivos que podem ocasionalmente precisar ser atualizados por diversas razões, incluindo correções de bugs pós-implantação, segurança, patches, introdução de novos recursos e melhorias de desempenho. Com isso, FOTA tem sido considerado mais ideal, especialmente quando se lida com implantação WSN/LPWAN inacessível fisicamente e em grande escala. Cenários onde os métodos manuais seriam muito caros ou apresentariam dificuldades físicas ou temporais (PULE; ABU-MAHFOUZ, 2019).

A atualização over-the-air pode ser vista como um método para programar código de computador em uma plataforma remota por meio de um dispositivo local. O método inclui receber o código de computador de tal modo que o código seja dividido em pacotes e são fornecidos por transmissão sem fio. O próximo passo envolve armazenar os pacotes de código em uma área alternativa de memória flash do dispositivo local de modo que os pacotes armazenados representem um código de computador. O dispositivo local então reconhece a recepção da cópia completa do código de computador e verifica a integridade do código recebido (QUADRI; SIDEK, 2014).

Como os dispositivos IoT serão sempre projetados para serem simples e acessíveis de forma a manter o seu tradicional design em tamanho reduzido, eles são inerentemente limitados por recursos em termos de poder computacional, energia/duração da bateria, comunicação largura

de banda e memória (PULE; ABU-MAHFOUZ, 2019).

É comum que dispositivos em uma WSN sejam dispositivos IoT atualmente, devido a troca de informações com entidades externas à rede. Esses dispositivos têm capacidade de processamento limitada, baixa capacidade de armazenamento e largura de banda de comunicação restrita. Essas limitações são devidas à energia limitada e ao tamanho físico dos dispositivos. Devido a essas restrições, é difícil empregar diretamente os mecanismos de segurança convencionais em dispositivos IoT. Para otimizar a segurança dos algoritmos, é importante entender as principais restrições dos dispositivos (SEN, 2010).

(SEN, 2010) categorizou as principais restrições de dispositivos IoT em restrições de energia, limitações de memória, comunicação não confiável, alta latência de comunicação e proteção física do dispositivo. Com relação ao consumo de energia, é possível separar a energia em três campos chave, sendo a energia utilizada pelo sensor transdutor, energia utilizada para comunicação e energia utilizada pelo microprocessador para executar instruções. Além disso, a fatores como segurança e latência estão atrelados ao consumo energético. Algoritmos de criptografia mais robustos exigem maior capacidade de processamento, consequentemente afetam o consumo energético. A proteção física do dispositivo é um grande desafio, visto que de acordo com a aplicação, o dispositivo pode ser instalado em uma região onde não é possível realizar o monitoramento físico do dispositivo, permitindo a prática de ações maliciosas como o tampering físico.

Além das restrições citadas acima, (ARAKADAKIS et al., 2021) apresentou o problema relacionado ao tempo de reinício de um dispositivo após a atualização e o impacto direto e indireto na rede de dispositivos. Após a instalação de uma nova versão de firmware, o dispositivo necessita ser reiniciado, então o bootloader executa o novo firmware. Isso pode introduzir uma sobrecarga significativa em aplicativos de IoT de tempo crítico, como aqueles usados nas áreas da aviação, saúde, carros inteligentes e assim por diante.

Sem a tecnologia over-the-air, os microcontroladores eram atualizados por interfaces físicas. Uma das interfaces mais utilizadas para configuração e testes é o JTAG. Essa interface surgiu em meados da década de 1980 para resolver o problema que os pacotes de circuitos integrados e a tecnologia de placas de circuito impresso evoluiu a ponto de testar circuitos usando sondas elétricas se tornar difícil ou impossível. Os pontos do circuito que precisavam ser acessados tornaram-se inacessíveis às sondas. A noção de varredura de limite permite o estado de um limite lógico de um circuito (o que teriam sido tradicionalmente pinos acessíveis a sondas) para serem lidos ou escritos em série através de pinos que são acessíveis. Hoje, as portas JTAG são amplamente utilizadas para fornecer um interface de depuração para processadores incorporados, permitindo um ambiente de depuração hospedado em PC para examinar e controlar o estado de um processador embarcado. A porta JTAG é usada, por exemplo, para ler o estado dos registradores do processador, para definir pontos de interrupção (breakpoints) em um programa e para executar linha a linha do programa em um processo de debug. Uma variante mais recente é

a depuração de fio serial (SWD), que fornece funcionalidade semelhante com menos pinos de conexão (LEE; SESHIA, 2016).

Um bootloader de inicialização é um aplicativo cujo objetivo principal é permitir que um software de sistema seja atualizado sem o uso de hardware especializado, como um programador JTAG. Em certos casos, também pode ser o primeiro ponto em que a integridade de um sistema embutido pode ser verificada. O bootloader gerencia as imagens dos sistemas. Existem diversos tipos e tamanhos de bootloaders. Eles podem se comunicar através de uma variedade de protocolos, como USART, CAN, I2C, Ethernet, USB e a lista continua para quantos protocolos existirem. Sistemas com gerenciadores de inicialização têm pelo menos duas imagens de programa coexistindo no mesmo microcontrolador e devem incluir código de ramificação que executa uma verificação para ver se uma tentativa de atualizar o software está em andamento (BENINGO, 2015). É importante levar em consideração que a implementação de um bootloader ocupa espaço de memória adicional em um microcontrolador. Existem abordagens onde o bootloader é integrado ao firmware e também em uma região de memória isolada para evitar problemas que possam inviabilizar a utilização do dispositivo.

A Internet das Coisas (IoT) apresenta-se como uma tecnologia emergente que é capaz de interligar um número massivo de dispositivos inteligentes heterogêneos para suportar complexos dados orientados a aplicações em diversos domínios, como cidades inteligentes, saúde, automação industrial, e assim por diante. É comum que os dispositivos que formam uma rede IoT operem sem vigilância por longos períodos em condições ambientais variáveis. Independente dos cuidados durante a fase de desenvolvimento, os dispositivos IoT podem precisar ser frequentemente reprogramados (ARAKADAKIS et al., 2021).

Levando em consideração algumas das aplicações citadas, é comum que diversos dispositivos eletrônicos permaneçam em regiões de difícil acesso. Com a utilização de um método de atualização FOTA, é possível realizar alterações em dispositivos sem a necessidade de acessá-lo fisicamente. A utilização de um método FOTA possui como vantagem o acesso ao dispositivo de forma remota, porém controlar quem o acessa remotamente é um grande desafio, o que contribui para que a segurança seja uma grande desvantagem para o método over-the-air.

Existem diversas áreas que, atualmente, estão desenvolvendo e realizando a utilização de métodos de atualização over-the-air.

No ramo automotivo, Ken Tindell publicou em seu blog (TINDELL, 2023) alguns problemas que podem ocorrer caso o método over-the-air seja adotado na indústria. O primeiro problema é a falha catastrófica, isto é, caso uma atualização possua uma falha, isso seria replicado em milhões de veículos nas ruas. A liberação de atualização por partes pode resolver esse problema. O segundo problema também cita uma falha, porém no processo de atualização, o que pode necessitar que o carro seja atualizado fisicamente para corrigir o problema. O terceiro e quarto problema citados por ele está relacionado a segurança da infraestrutura e a segurança do processo de gravação do firmware no dispositivo. A segurança está sendo abordada em

parte pela indústria automotiva, por meio de um módulo de segurança de hardware (HSM). A indústria definiu o padrão Secure Hardware Extensions (SHE) e os chips projetados para aplicações automotivas incluem esses módulos. Eles fornecem não apenas funções criptográficas (criptografia, autenticação, números aleatórios), mas também distribuição e armazenamento seguros de chaves. Para veículos rodoviários, a ISO 26262 rege as regras para o funcionamento seguro dos veículos relacionado às partes elétricas e eletrônicas.

2.8 Infraestrutura de rede para IoT

A Internet das Coisas, ou IoT, é formada por uma enorme diversidade de dispositivos diferentes que funcionam de maneira autônoma, produzindo e consumindo dados nas camadas mais distantes das redes e protocolos. Para projetar sistemas de comunicação nesse contexto é fundamental compreender as limitações e desafios que surgem, já que não se trata apenas de conectar máquinas, mas de integrar tecnologias muito distintas em um mesmo ecossistema. A IoT combina redes pessoais, como aquelas usadas em dispositivos vestíveis e sensores próximos ao usuário, redes locais, como o Wi-Fi em casas e empresas, e redes de longa distância, como LoRaWAN, Narrow Band-IoT ou 5G, que cobrem grandes áreas. Todas essas redes se entrelaçam em uma teia de canais de comunicação, permitindo que os dispositivos troquem informações de forma contínua e inteligente, criando um ambiente conectado que vai do nível mais próximo do usuário até infraestruturas globais (LEA, 2018).

A infraestrutura de rede para IoT consiste na tecnologia utilizada para realizar a comunicação entre os dispositivos. Os dispositivos podem conectar-se entre si em uma região de baixa distância ou longas distâncias.

Como existem muitas tecnologias sem fio dentro do ecossistema da Internet das Coisas, cada uma delas apresenta especificações e benefícios próprios. Algumas oferecem maior alcance, outras priorizam baixo consumo de energia, enquanto certas soluções entregam altas taxas de transmissão de dados. Por isso, é difícil afirmar que exista uma tecnologia “perfeita” para todos os casos (AL-SARAWI et al., 2017).

Além das tecnologias compatíveis com TCP/IP, a LoRa/LoRaWAN vem ganhando destaque no cenário de IoT por seu foco em baixo consumo de energia, possibilitando a operação de dispositivos por longos períodos com baterias de pequena capacidade.

2.8.1 LoRa/LoRaWAN

LoRaWAN não foi concebido como uma pilha completa de rede dentro do modelo OSI e, por isso, não incorpora diversas funcionalidades comuns em redes LTE, como roaming, mecanismos de retransmissão, qualidade de serviço e gerenciamento de desconexões. Esses recursos precisam ser adicionados pelos desenvolvedores ou integradores quando necessários.

Além disso, a arquitetura depende de uma interface em nuvem, o que implica em algum ponto da cadeia de valor a necessidade de uma assinatura de serviços.

O protocolo de acesso utilizado pelo LoRaWAN é o ALOHA, que aumenta a complexidade na validação e no reconhecimento de mensagens e contribui para taxas de erro elevadas, podendo ultrapassar 50%. A capacidade de comunicação no sentido descendente ainda é limitada, tornando o protocolo essencialmente unidirecional. Isso pode ser suficiente em alguns casos de uso, mas reduz a flexibilidade. O sistema apresenta alta latência, não suporta aplicações em tempo real e não oferece atualizações de firmware OTA.

A mobilidade também é um desafio, já que mensagens relativamente pequenas, de 40 a 60 bytes, podem levar mais de um segundo para serem transmitidas, o que se torna problemático em dispositivos em movimento, como veículos em alta velocidade. Além disso, as estações base exigem condições de tempo invariantes para que o sinal não altere o tempo de propagação. Em termos de geolocalização, a precisão é limitada a cerca de 100 metros, obtida por medições de intensidade de sinal ou tempo de voo. A triangulação com três ou mais estações base pode melhorar a acurácia, mas ainda assim permanece restrita em comparação com outras tecnologias (LEA, 2018).

LoRaWAN apresenta desafios importantes relacionados à escalabilidade, interferência e segurança. Estudos mostram que, em implantações urbanas, é recomendável utilizar a maior taxa de transmissão possível para otimizar a capacidade, embora a ocupação dos terminais por área não se altere significativamente em velocidades menores. No aspecto de interferência, o uso do esquema ALOHA, sem gerenciamento específico de acesso ao espectro, resulta em transmissões cegas que aumentam a probabilidade de colisões e perdas de pacotes. Já em relação à segurança, o rápido crescimento das conexões expõe vulnerabilidades difíceis de mitigar, pois as características físicas da tecnologia permitem ataques complexos e a exigência de alta eficiência energética limita a adoção de defesas robustas. Técnicas de segurança na camada física podem oferecer proteção, mas ainda carecem de robustez suficiente, e os métodos atuais de geração de chaves frequentemente restringem a comunicação segura a apenas duas partes válidas (KAMAL et al., 2023).

2.8.2 TCP/IP

O uso da pilha TCP/IP em sistemas de IoT consome recursos, mas traz benefícios essenciais. Independentemente do protocolo usado nos sensores, os dados acabam sendo enviados para nuvens públicas, privadas ou híbridas, e fora das redes pessoais o mundo já é baseado em TCP/IP. Essa tecnologia se destaca por sua ubiquidade, já que está presente em praticamente todos os sistemas e meios de comunicação; por sua longevidade, com mais de quatro décadas de uso contínuo; por ser padronizada; por sua escalabilidade, capaz de suportar bilhões de dispositivos e endereços únicos com IPv6; e por sua confiabilidade, oferecendo entrega de pacotes em modelo best-effort. Além disso, há ampla disponibilidade de ferramentas para gerenciar redes IP, o que

facilita a construção, manutenção e expansão de sistemas IoT (LEA, 2018).

As tecnologias sem fio que apresentam grande utilização para dispositivos IoT e baseiam-se na pilha TCP/IP são redes de telefonia celular e o Wi-Fi.

Neste trabalho, utilizou-se a pilha TCP/IP por meio do protocolo Wi-Fi para a comunicação, escolhida pelos motivos de compatibilidade e escalabilidade. A pilha TCP/IP possibilita a conexão com outros dispositivos através da internet, o que facilita o acesso a servidores de arquivos. Além disso, o throughput dessa abordagem oferece ampla capacidade para o processo de atualização OTA, considerando que os firmwares geralmente possuem tamanhos na ordem de quilobytes a megabytes. Para as implementações deste trabalho, os arquivos binários variam entre 700KB e 1000KB.

2.9 FreeRTOS

FreeRTOS é um kernel de sistema operacional em tempo real (RTOS) de código aberto que está integrado ao ESP-IDF como um componente. Assim, todas as aplicações ESP-IDF e muitos componentes ESP-IDF são escritos com base no FreeRTOS (ESPRESSIF, 2023).

2.9.1 Tasks

Uma aplicação em tempo real que utiliza um RTOS pode ser estruturada como um conjunto de tarefas independentes. Cada tarefa é executada dentro de seu próprio contexto, sem nenhuma dependência coincidente de outras tarefas dentro do sistema ou do próprio agendador RTOS. Apenas uma tarefa dentro da aplicação pode ser executada a qualquer momento e a decisão de qual tarefa será executada fica a critério do agendador RTOS. O agendador RTOS pode, portanto, iniciar e parar repetidamente cada tarefa à medida que o aplicativo é executado. Como uma tarefa não tem conhecimento da atividade do agendador RTOS, é responsabilidade do agendador RTOS garantir que o contexto do processador (valores de registro, conteúdo da pilha, etc.) quando uma tarefa é trocada, seja exatamente igual ao momento que ocorreu a troca. Para conseguir isso, cada tarefa recebe sua própria pilha. Quando a tarefa é trocada, o contexto de execução é salvo na pilha dessa tarefa para que também possa ser restaurado exatamente quando a mesma tarefa for posteriormente trocada novamente. O diagrama de estados das tarefas no FreeRTOS define os possíveis estados e as transições entre eles que as tarefas podem realizar juntamente com o escalonador (BARRY, 2016). Conforme Figura 4.

A prioridade das tarefas define a frequência na qual uma tarefa será executada em relação às outras tarefas. As prioridades podem ser atribuídas de 0, que é a prioridade mais baixa, a (`config_MAXPRIORITIES - 1`), que é a maior prioridade. É importante salientar que as prioridades não devem ser diferentes caso as tarefas não possuam estruturas internas que as coloquem em estado bloqueado, pois nesse caso, uma tarefa com um valor de prioridade maior

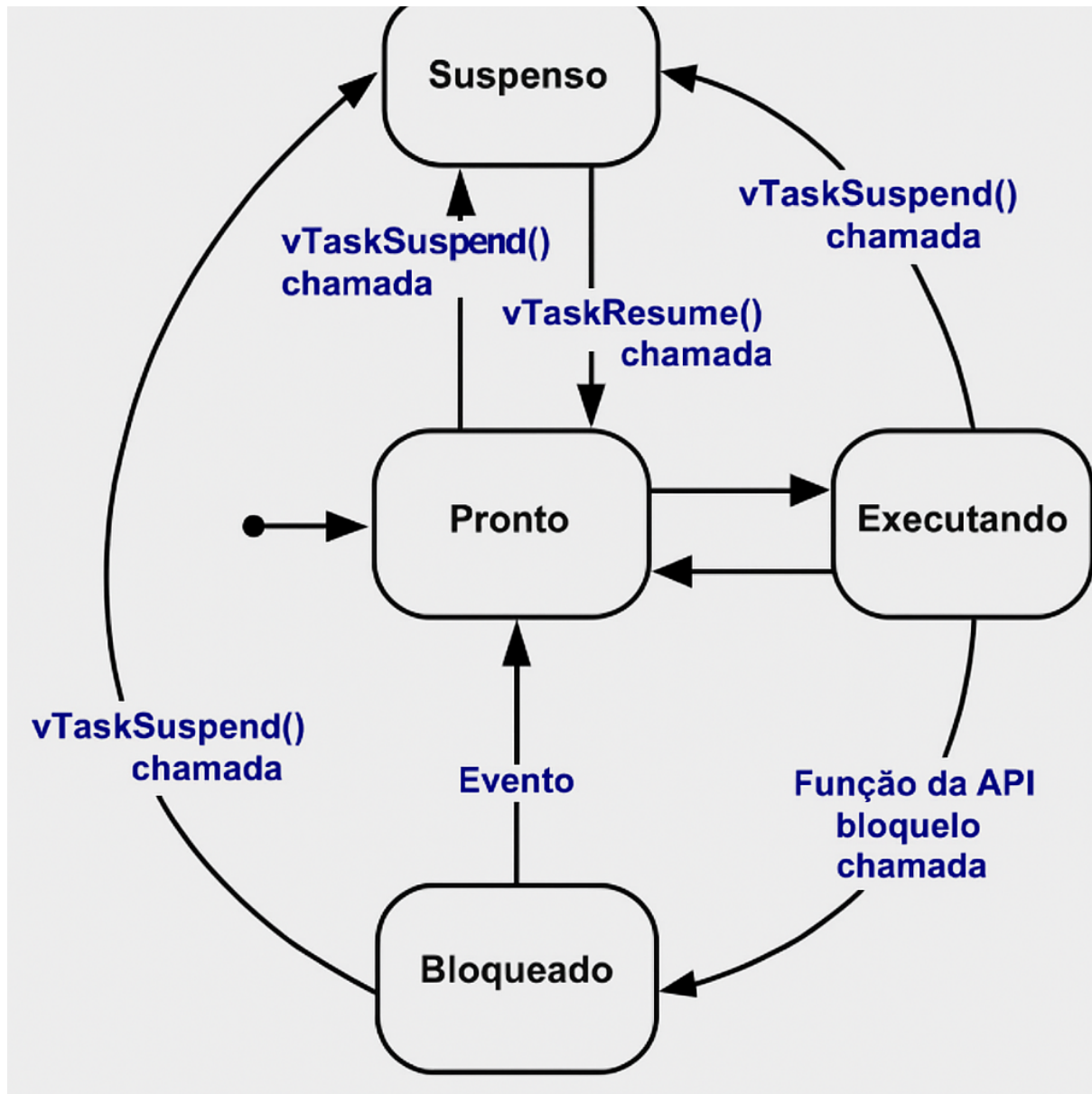


Figura 4 – Diagrama de estados FreeRTOS. Adaptado de: (FREERTOS, 2025b)

pode fazer a utilização total do tempo do processador e ocasionar o problema de starving para as outras tarefas (FREERTOS, 2025a).

2.9.2 Event Groups

Um grupo de eventos (event group) é um conjunto de bits de eventos. Bits de eventos individuais dentro de um grupo de eventos são referenciados por um número de bit (FREERTOS, 2025a).

1. O bit de evento que significa "Uma mensagem foi recebida e está pronta para processamento" pode ser o bit número 0 dentro de um grupo de eventos.

2. O bit de evento que significa "O aplicativo enfileirou uma mensagem que está pronta para ser enviada a uma rede" pode ser o bit número 1 dentro do mesmo grupo de eventos.
3. O bit de evento que significa "É hora de enviar uma mensagem de pulsação para uma rede" pode ser o bit número 2 dentro do mesmo grupo de eventos.

3 Metodologia

3.1 Cronograma de atividades do TFG

F.0: Planejamento e Fundamentação

Nesta fase, é necessário construir as bases teóricas e metodológicas do trabalho. Pesquisar o estado da arte permite entender as soluções existentes para FOTA, suas aplicações e desafios. Reunir bibliografia auxilia a consolidar um referencial teórico sólido, enquanto decidir o dispositivo para implementação garante que a tecnologia seja aplicada conforme a base teórica.

Atividades planejadas para a fase 0

- A.0.1 Pesquisa do Estado da Arte
- A.0.2 Reunião Bibliográfica
- A.0.3 Decisão do Dispositivo para Implementação

Resultados esperados para a fase 0

Mapear as tecnologias FOTA existentes e suas principais vantagens/desvantagens.
 Coletar referências técnicas e científicas para embasar a pesquisa.
 Definir o dispositivo adequado para testes e implementação da solução FOTA.
 Elaborar um plano detalhado para execução da segunda etapa, incluindo requisitos e metodologia.

F.1: Desenvolvimento e Escrita

Com base na pesquisa realizada, devem ser realizadas a implementação e o desenvolvimento da monografia. Implementar a tecnologia FOTA abordando os principais conceitos adquiridos pela pesquisa a fim de estabelecer requisitos para a implementação da tecnologia em geral, testar a atualização remota, garantir a segurança do firmware e analisar os impactos de desempenho e segurança. Além da redação, efetuar a revisão.

Atividades planejadas para a fase 1

- A.1.1 Elaboração da Monografia
- A.1.2 Implementação
- A.1.3 Validação e Ajustes Finais

Resultados esperados para a fase 1

Implementar a funcionalidade de atualização FOTA no dispositivo escolhido.
 Comprometer segurança propositalmente para justificar a necessidade das implementações.
 Documentar detalhadamente o processo e desafios encontrados.
 Estruturar a monografia, apresentando resultados, conclusões e trabalhos futuros.

2025					
01	02	03	04	05	06
	F.0: A.0.1				
		F.0: A.0.2			
		F.0: A.0.3			
				F.0: A.1.1	
				F.0: A.1.2	
					F.0: A.1.3

3.2 Configuração do ambiente

3.2.1 Introdução

Existem duas principais abordagens para o desenvolvimento com o microcontrolador ESP32: o framework Arduino Core e o framework ESP-IDF (Espressif IoT Development Framework). A utilização do Arduino Core implica na adoção de abstrações típicas da linguagem Arduino, facilitando o desenvolvimento, porém limitando o controle sobre aspectos específicos do hardware. Neste trabalho, opta-se pela utilização exclusiva do ESP-IDF, devido à sua maior granularidade e compatibilidade com recursos nativos do ESP32, permitindo uma abordagem mais robusta e profissional. O ESP32 oferece suporte nativo à atualização de firmware via protocolo HTTP/HTTPS por meio de bibliotecas proprietárias. Dentre elas, destaca-se a biblioteca ESP HTTPS OTA, a mais recente solução disponibilizada pela Espressif, que fornece funcionalidades para a realização de atualizações remotas seguras. Além disso, o ESP32 incorpora diversos mecanismos avançados de segurança embarcada, incluindo inicialização segura, criptografia de armazenamento e gerenciamento de chaves por meio de módulos dedicados de hardware. Essas características tornam o ESP32 uma plataforma adequada para a implementação da tecnologia FOTA, estabelecendo parâmetros técnicos e requisitos de segurança que devem ser considerados por dispositivos que visam adotar tal tecnologia. As implementações do trabalho são baseadas nos materiais e exemplos fornecidos pela Espressif ([ESPRESSIF, 2023](#)).

Serão desenvolvidas duas abordagens para a atualização de firmware OTA (Over-the-Air). A primeira utilizará os protocolos HTTP/HTTPS para realizar a transferência do firmware. A segunda proposta acrescenta mecanismos de verificação de integridade e autenticidade, por meio da utilização de assinatura digital do código, visando aumentar a segurança do processo de atualização e diminuir a superfície de ataque malicioso.

3.2.2 Características do ambiente e dispositivo de teste

A [Tabela 1](#) demonstra os materiais e sistemas utilizados para realizar a implementação da tecnologia FOTA de acordo com as abordagens tratadas no presente trabalho.

Material	Sistema / Versão
Placa de desenvolvimento ESP32	ESP32-WROOM-32
Cabo USB	USB 2.0
Computador	Windows 11 24H2
Editor de Código/IDE	VS Code 1.94
Interpretador Python	Python 3.12.5
Framework para ESP32	ESP-IDF 5.4.1
Conexão de Internet	Wi-Fi residencial 2.4 GHZ
Servidor de arquivos	HTTP/HTTPS

Tabela 1 – Lista de materiais utilizados na implementação. Fonte: Dados da pesquisa.

O ambiente de desenvolvimento é composto por um computador, um editor de código ou IDE, o interpretador Python e o framework ESP-IDF. Nesses dispositivos, são realizadas a implementação e configuração do sistema. A transferência das configurações, bootloader e firmware para a placa de desenvolvimento ESP32 ocorre por meio de um cabo USB. Uma vez programado, o dispositivo ESP32 contém o bootloader e o firmware disponíveis para inicialização. Após a inicialização, o dispositivo está disponível para se conectar ao Wi-Fi e buscar por atualizações, utilizando como mecanismos de transmissão os protocolos de camada de aplicação HTTP ou HTTPS. O servidor de arquivos hospeda o firmware e um arquivo de configuração JSON. O arquivo de configuração contém a versão do firmware e o endereço para acessá-lo e será mostrado posteriormente no documento.

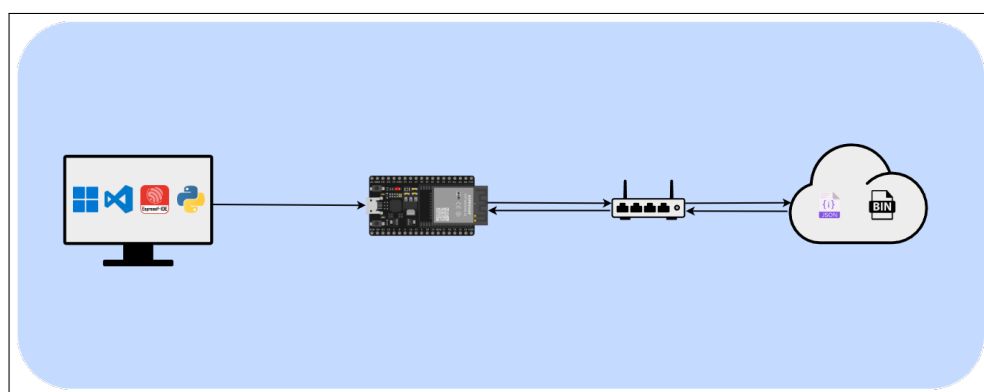


Figura 5 – Diagrama das entidades envolvidas na implementação. Fonte: Elaborado pelo autor.

3.2.3 Detalhes de implementação

Antes de iniciar a implementação, é importante validar questões relacionadas ao tamanho de memória flash disponível e tabela de partições do dispositivo. Por meio do editor de configuração "menuconfig", é possível alterar configurações do sistema através de uma interface com

opções pre-estabelecidas pela Espressif em vez de definir parâmetros manualmente no arquivo "sdkconfig" do projeto. A interface facilita o processo para definir parâmetros de configuração no arquivo de configuração do projeto e foi utilizada durante as implementações demonstradas no trabalho. O ESP32 possui um acervo de certificados confiáveis para entidades conhecidas, como o Github, que foi utilizado no projeto para hospedar o arquivo binário. Desta forma, não há a necessidade de criar um certificado digital dedicado, sendo esta etapa recomendada em um cenário de produção. A utilização se dá por meio da configuração "*Enable trusted root certificate bundle*".

□

Figura 6 – Exemplo da interface do menu de configurações. Fonte: Elaborado pelo autor.

Através do editor de configuração demonstrado na [Figura 6](#), algumas opções foram modificadas para o projeto desenvolvido no trabalho. Todas as alterações realizadas estão descritas abaixo.

- Flash Size: utilizada para definir o tamanho da flash do dispositivo, dentro do recurso disponível na placa utilizada. Para o projeto, foram utilizados 4MiB;
- HTTPS: utilizada para definir as estruturas de conexão e autenticação via HTTPS;
- Allow HTTP for OTA: permite a instalação de firmware OTA via HTTP;
- Partition Table: define como a memória flash será dividida. Neste caso, utilizando o modo "Factory app, two OTA definitions";
- Offset of partition table: endereço utilizado para definir a tabela de partições. Se o tamanho do bootloader exceder o limite, será necessário alterar o deslocamento da tabela de partição;
- Enable trusted root certificate bundle: o ESP32 utiliza uma lista de certificados previamente carregada para utilizar com ESP32 e a autenticação TLS;
- Require signed app images: define a necessidade de binários assinados;
- Bootloader verifies app signatures: verifica a assinatura no binário antes de inicializá-lo;
- Verify app signature on update: habilita a verificação de assinatura de código quando atualizando o dispositivo;
- Sign binaries during build: efetua a assinatura do binário automaticamente no processo de build;
- Secure boot private signing key: define o nome do certificado utilizado para a assinatura de código;

- `configGENERATE_RUN_TIME_STATS`: habilita estruturas do FreeRTOS para obter estatísticas na execução das tarefas;
- `Enable app rollback support`: habilita a verificação de firmware e o processo de rollback em caso de falha.

É possível dividir a memória flash por meio de configurações pré estabelecidas pelo fabricante ou criar uma tabela customizada por meio de um arquivo CSV. Para a implementação deste trabalho, será utilizada a tabela de partições com duas partições dedicadas a aplicações OTA. Neste caso, os espaços para os firmwares possuem 1MiB cada. A [Figura 7](#) apresenta o arquivo CSV para o esquema de partição padrão do dispositivo. Como o projeto utiliza a funcionalidade OTA, a [Figura 8](#) demonstra o CSV utilizado para o projeto desenvolvido durante o trabalho.

```
# ESP-IDF Partition Table
# Name,      Type, SubType, Offset,  Size, Flags
nvs,         data, nvs,      0x9000,  0x6000,
phy_init,    data, phy,       0xf000,  0x1000,
factory,     app,  factory, 0x10000, 1M,
```

Figura 7 – Tabela de partições padrão ESP32. Fonte: ([ESPRESSIF, 2023](#)).

```
# ESP-IDF Partition Table
# Name,      Type, SubType, Offset,  Size, Flags
nvs,         data, nvs,      0x9000,  0x4000,
otadata,     data, ota,       0xd000,  0x2000,
phy_init,    data, phy,       0xf000,  0x1000,
factory,     app,  factory, 0x10000, 1M,
ota_0,       app,  ota_0,    0x110000, 1M,
ota_1,       app,  ota_1,    0x210000, 1M,
```

Figura 8 – Tabela de partições para duas definições OTA do ESP32. Fonte: ([ESPRESSIF, 2023](#)).

Como o dispositivo utilizado foi configurado para utilizar 4MiB de memória flash, isto é, 4,194,304 bytes ou 1,048,576 words, levando em consideração que o ESP32 possui um processador com arquitetura capaz de manipular dados de 32 bits. De maneira visual, a [Figura 9](#) permite entender a organização da memória flash do dispositivo. As partições contidas na tabela de partições utilizada no projeto são descritas a seguir.

1. bootloader (24576 bytes) - Inicializador do dispositivo e seus recursos.
2. partition table (4096 bytes) - Definição das separações do espaço de memória.
3. nvs (16384 bytes) - Utilizada pela biblioteca de armazenamento de chave/valor na memória flash.
4. otadata (8192 bytes) - Arquivos necessários para realizar a implementação da funcionalidade OTA.
5. phyinit (4096 bytes) - Dados de inicialização do dispositivo PHY.
6. factory (1048576 bytes) - Espaço primário de software do dispositivo. Deve ser gravado por meio físico e é utilizado para rollback.
7. ota0 (1048576 bytes) - Partição 0 para software carregado via OTA.
8. ota1 (1048576 bytes) - Partição 1 para software carregado via OTA.

É importante deixar claro que para a variação de implementação utilizando assinatura de código, o bootloader sofreu alteração de tamanho, o que modifica os endereços mostrados anteriormente. Na [subseção 3.3.2](#) serão abordadas as modificações necessárias para realizar a implementação.

3.3 Implementações

As implementações foram realizadas a partir do projeto de exemplo "blink_example" do ESP-IDF. As modificações foram baseadas no GitHub fornecido pela Espressif com diversas implementações para o ESP32 ([ESPRESSIF, 2025](#)). Para todas as implementações, um arquivo JSON foi utilizado para informações sobre o arquivo binário utilizado na atualização. A utilização de um arquivo de configuração auxilia o controle da atualização por meio de parâmetros que podem ser utilizados para modificar como a atualização de firmware será realizada. A estrutura do projeto de implementação base é exposta na [Figura 10](#).

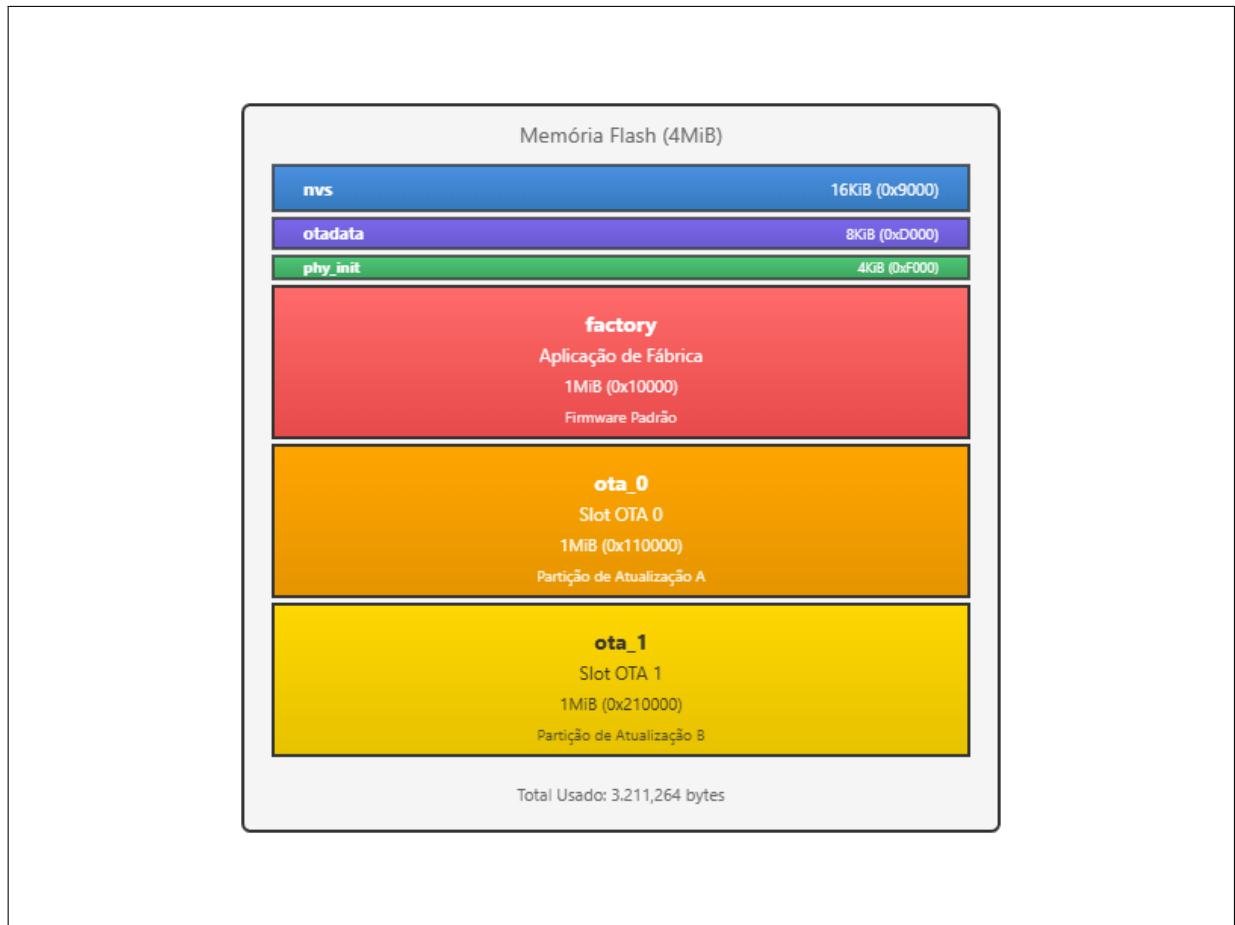


Figura 9 – Organização da memória flash com duas definições OTA. Fonte: Elaborado pelo autor.

3.3.1 Código Fonte Base

O código a ser descrito nessa seção apresenta todas as estruturas necessárias para realizar o processo de atualização OTA no ESP32 utilizando o HTTP/HTTPS. Antes de analisar o código, é importante entender a estrutura do projeto. O código fonte do firmware OTA apresenta três arquivos de código em C e 2 arquivos de cabeçalho.

1. `wifi_sta.c`: arquivo com as funções destinadas a conectar o dispositivo ESP32 ao Wi-Fi;
2. `wifi_sta.h`: arquivo de cabeçalho com a declaração das funções do `wifi_sta.c`;
3. `ota_functions.c`: arquivo com as funções destinadas a realizar a atualização OTA;
4. `ota_functions.h`: arquivo de cabeçalho com a declaração das funções do `ota_functions.c`;
5. `main.c`: arquivo com a criação das tarefas, loops e execução das funções.

Para iniciar o processo de atualização, é necessário analisar o arquivo JSON com as informações necessárias para realizar o processo de atualização OTA. A chave "version" define

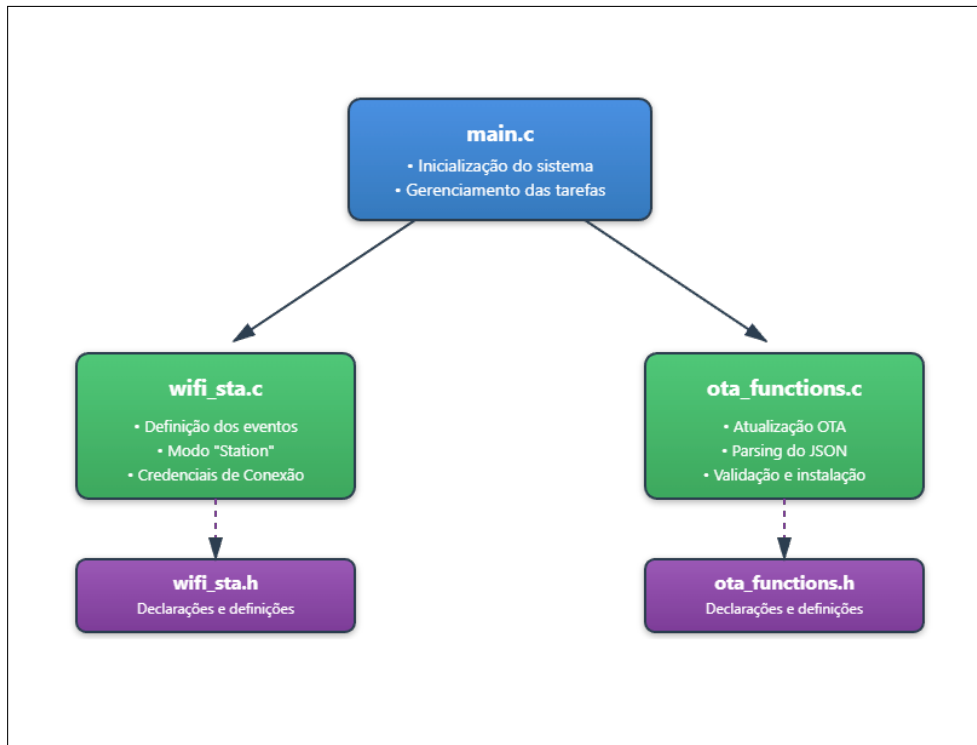


Figura 10 – Estrutura do projeto de implementação. Fonte: Elaborado pelo autor.

uma versão para o binário que pode ser baixado pelo dispositivo. O seu valor é utilizado para comparar com a versão atual do firmware presente no dispositivo. A chave "URL" define endereço a ser utilizado para descarregar o arquivo binário através da conexão HTTP/HTTPS.

```

{
  "version": "2.0",
  "url":
    "https://github.com/Felipeds97/otaupdate/raw/refs/heads/main/
    blink_only.bin"
}
  
```

As funções presentes no projeto e suas finalidades serão mostradas a seguir.

1. (`_http_event_handler`) Efetua o tratamento dos eventos do tipo HTTP.

```

esp_err_t _http_event_handler(esp_http_client_event_t *evt) {
  return ESP_OK;
}
  
```

2. (`perform_ota`) Realiza o processo de atualização OTA no ESP32.

```

void perform_ota(const char *url) {
  esp_http_client_config_t http_config = {
  
```



```

        .url = url,
        .event_handler = _http_event_handler,
        //comment two lines below and set allow ota http in
        menuconfig to skip https validation
        //.use_global_ca_store = true,
        //.crt_bundle_attach = esp_crt_bundle_attach
    };

    esp_https_ota_config_t ota_config = {
        .http_config = &http_config,
        //.partial_http_download = false, // Optional: set true
        if needed
    };

    esp_err_t ret = esp_https_ota(&ota_config);
    if (ret == ESP_OK) {
        ESP_LOGI(TAG, "OTA successful! Rebooting...");
        esp_restart();
    } else {
        ESP_LOGE(TAG, "OTA failed: %s", esp_err_to_name(ret));
    }
}

bool is_newer_version(const char *new_version) {
    return strcmp(new_version, VERSION_CURRENT) > 0; //
        Simplified comparison
}

```

3. (is_newer_version) Efetua a comparação entre a versão instalada e a versão disponível para atualização.

```

bool is_newer_version(const char *new_version) {
    return strcmp(new_version, VERSION_CURRENT) > 0; //
        Simplified comparison
}

```

4. (check_for_update) Efetua o descarregamento e parsing do JSON. Inicializa o processo de atualização OTA.

```

void check_for_update() {
    //ESP_ERROR_CHECK(esp_tls_init_global_ca_store());
}

```

```
esp_http_client_config_t config = {
    .url = UPDATE_URL,
    .event_handler = _http_event_handler,
    //comment two lines below and set allow ota http in
    //menuconfig to skip https validation
    //.use_global_ca_store = true,
    //.crt_bundle_attach = esp_crt_bundle_attach,
};

esp_http_client_handle_t client =
    esp_http_client_init(&config);
if (client == NULL) {
    ESP_LOGE(TAG, "Failed to initialize HTTP client");
    return;
}

// Open the HTTP connection
esp_err_t err = esp_http_client_open(client, 0);
if (err != ESP_OK) {
    ESP_LOGE(TAG, "Failed to open HTTP connection: %s",
        esp_err_to_name(err));
    esp_http_client_cleanup(client);
    return;
}

int len = esp_http_client_fetch_headers(client);
if (len <= 0) {
    ESP_LOGE(TAG, "Invalid content length: %d", len);
    esp_http_client_cleanup(client);
    return;
}

char *buffer = malloc(len + 1);
if (!buffer) {
    ESP_LOGE(TAG, "Failed to allocate memory for JSON
        buffer");
    esp_http_client_cleanup(client);
    return;
}

int total_read = 0;
```

```

int read_len = 0;
while ((read_len = esp_http_client_read(client, buffer +
    total_read, len - total_read)) > 0) {
    total_read += read_len;
}
buffer[total_read] = '\0';

//ESP_LOGI(TAG, "Raw JSON: %s", buffer);

cJSON *json = cJSON_Parse(buffer);
if (json) {
    const cJSON *ver = cJSON_GetObjectItemCaseSensitive(json,
        "version");
    const cJSON *url = cJSON_GetObjectItemCaseSensitive(json,
        "url");
    if (cJSON_IsString(ver) && cJSON_IsString(url)) {
        ESP_LOGI(TAG, "Latest version: %s", ver->valuestring);
        if (is_newer_version(ver->valuestring)) {
            ESP_LOGI(TAG, "New version found! Starting OTA...");
            perform_ota(url->valuestring);
        } else {
            ESP_LOGI(TAG, "Already up to date.");
        }
    }
    cJSON_Delete(json);
}

free(buffer);
esp_http_client_cleanup(client);
}

```

5. (wifi_event_handler) Efetua o tratamento dos eventos do Wi-Fi.

```

static void wifi_event_handler(void* arg, esp_event_base_t
    event_base,
                                int32_t event_id, void* event_data) {
if (event_base == WIFI_EVENT && event_id ==
    WIFI_EVENT_STA_START) {
    esp_wifi_connect();
} else if (event_base == WIFI_EVENT && event_id ==
    WIFI_EVENT_STA_DISCONNECTED) {
    ESP_LOGW(TAG, "Disconnected from Wi-Fi. Reconnecting...");
}

```

```

    esp_wifi_connect();
} else if (event_base == WIFI_EVENT && event_id ==
    WIFI_EVENT_STA_CONNECTED) {
    ESP_LOGW(TAG, "Wi-Fi connected.");
} else if (event_base == IP_EVENT && event_id ==
    IP_EVENT_STA_GOT_IP) {
    ip_event_got_ip_t *event = (ip_event_got_ip_t
        *)event_data;
    ESP_LOGI(TAG, "Got IP: " IPSTR,
        IP2STR(&event->ip_info.ip));
    xEventGroupSetBits(s_wifi_event_group,
        WIFI_CONNECTED_BIT);
}
}

```

6. (wifi_init_sta) Realiza os procedimentos necessários para inicializar o dispositivo Wi-Fi do ESP32.

```

void wifi_init_sta(void) {

    s_wifi_event_group = xEventGroupCreate();

    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());

    esp_netif_create_default_wifi_sta();

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    ESP_ERROR_CHECK(esp_event_handler_register(WIFI_EVENT,
        ESP_EVENT_ANY_ID, &wifi_event_handler, NULL));
    ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT,
        IP_EVENT_STA_GOT_IP, &wifi_event_handler, NULL));

    wifi_config_t wifi_config = {
        .sta = {
            .ssid = WIFI_SSID,
            .password = WIFI_PASSWORD,
            .threshold.authmode = WIFI_AUTH_WPA2_PSK,
            .pmf_cfg = {
                .capable = true,

```

```

        .required = false
    },
},
};

ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA,
    &wifi_config));
ESP_ERROR_CHECK(esp_wifi_start());

ESP_LOGI(TAG, "Connecting to Wi-Fi network: %s", WIFI_SSID);

EventBits_t bits = xEventGroupWaitBits(s_wifi_event_group,
    WIFI_CONNECTED_BIT,
    pdFALSE,
    pdFALSE,
    pdMS_TO_TICKS(10000));

while(1) {
    if (bits & WIFI_CONNECTED_BIT) {
        ESP_LOGI(TAG, "Wi-Fi connected successfully");
        break;
    }
}
}

```

7. (ota_update) Tarefa para realizar a atualização OTA.

```

void ota_update(void *pvParameter) {
    while(1) {
        ESP_LOGI(TAG, "Starting OTA version check...");
        check_for_update();
        vTaskDelay(60000 / portTICK_PERIOD_MS);
    }
}

```

8. (blink_task) Tarefa para piscar LED.

```

void blink_task(void *pvParameter) {

    ESP_LOGI(TAG, "Example configured to blink GPIO LED!");
}

```

```

gpio_reset_pin(BLINK_GPIO);
gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
while(1) {
    gpio_set_level(BLINK_GPIO, 0);
    vTaskDelay(2000 / portTICK_PERIOD_MS);
    gpio_set_level(BLINK_GPIO, 1);
    vTaskDelay(2000 / portTICK_PERIOD_MS);
}
}

```

9. (monitor_task) Tarefa para monitorar a utilização de recursos.

```

void monitor_task(void *pvParameters) {
    char stats[2048];
    //int count = 0; if you need to count iterations (* times
    //the task is in running state)
    while (1) {
        vTaskGetRunTimeStats(stats);
        ESP_LOGI("MONITOR", "\nTask CPU Usage:\n%s", stats);
        UBaseType_t watermark =
            uxTaskGetStackHighWaterMark(monitor_task_handle);
        // NULL = current task
        printf("Stack high watermark: %u words\n", watermark);
        //ESP_LOGI("iteration", "%d", ++count);
        vTaskDelay(60000 / portTICK_PERIOD_MS);
    }
}

```

10. (app_main) Função principal com a criação das tarefas e inicialização dos periféricos.

```

void app_main(void)
{
    //Initialize NVS
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret ==
        ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ret = nvs_flash_init();
    }
    ESP_ERROR_CHECK(ret);
}

```

```
wifi_init_sta();

// After peripherals startup, you can set rollback
// validation, only use if you need rollback
/*
esp_err_t ota_ret = esp_ota_mark_app_valid_cancel_rollback();
if (ota_ret == ESP_OK) {
    ESP_LOGI(TAG, "Firmware marked as valid, rollback
    canceled.");
} else {
    ESP_LOGE(TAG, "Failed to mark firmware valid: %s",
    esp_err_to_name(ota_ret));
}
*/
xTaskCreate(&blink_task, "blink_task", 2048, NULL, 5, NULL);
xTaskCreate(&ota_update, // Pointer to your task function
    "ota_update", // Task name (for debugging)
    8196, // Stack size in **words** 4 bytes for
    ESP32
    NULL, // Task parameter
    5, // Priority
    &monitor_task_handle); // Optional task handle
    (NULL if unused)
xTaskCreate(&monitor_task, "monitor_task", 4096, NULL, 5,
    NULL);
}
```

Para visualizar o projeto e utilizá-lo, consulte o [Apêndice A](#).

3.3.2 Variação com Assinatura de Código

Embora a utilização do protocolo HTTPS para a transmissão do firmware tenha garantido a autenticidade, isto é, o receptor e o emissor são capazes de se identificar, a integridade do arquivo binário ainda permanece comprometida. Utilizando a abordagem exposta na [subseção 3.3.1](#), um firmware envenenado propositalmente foi inserido no servidor de arquivos, partindo da ideia que o atacante obteve acesso ao servidor que hospeda o arquivo JSON ou o arquivo binário. Neste caso, sem a assinatura de código, o firmware seria baixado e executado pelo dispositivo.

Para garantir a integridade do firmware a ser utilizado para a atualização OTA, foi

adicionada a implementação da assinatura de código com um certificado digital no firmware a ser utilizado para a atualização OTA. Além disso, o ESP32 foi configurado para adicionar a funcionalidade de verificação no processo de inicialização.

Com relação ao código fonte apresentado na [subseção 3.3.1](#), não foram necessárias alterações, sendo a alteração realizada apenas nas configurações do projeto através da interface mostrada na [Figura 6](#) e pontualmente descritas a seguir.

- Offset of partition table: endereço utilizado para definir a tabela de partições. Se o tamanho do bootloader exceder o limite, será necessário alterar o deslocamento da tabela de partição;
- Require signed app images: define a necessidade de binários assinados;
- Bootloader verifies app signatures: verifica a assinatura no binário antes de inicializá-lo;
- Verify app signature on update: habilita a verificação de assinatura de código quando atualizando o dispositivo;
- Sign binaries during build: efetua a assinatura do binário automaticamente no processo de build;
- Secure boot private signing key: define o nome do certificado utilizado para a assinatura de código;

A tabela de partições teve o endereço de início alterado para 0x10000 (32KiB de deslocamento) devido ao aumento do tamanho do bootloader conforme [Figura 11](#)

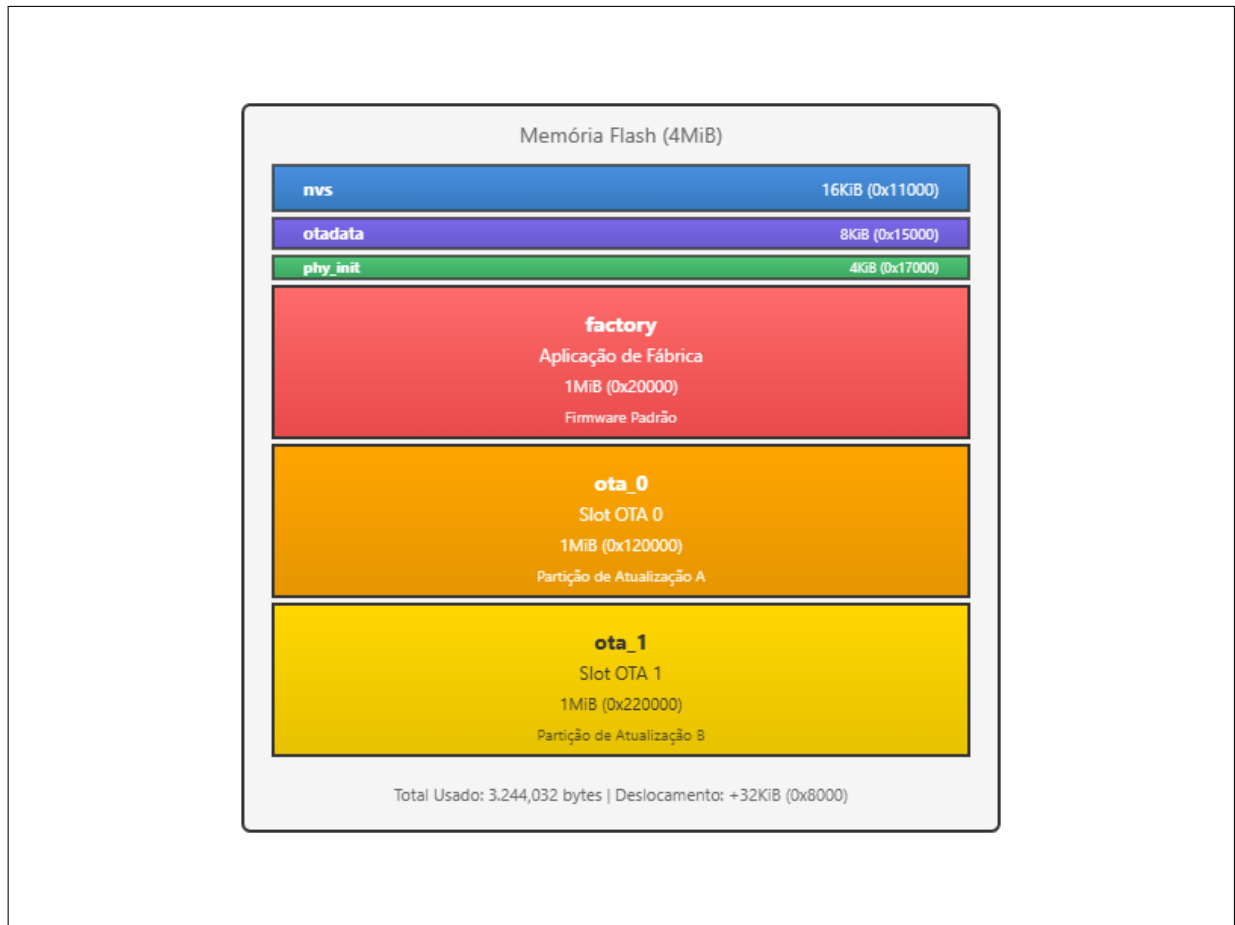


Figura 11 – Organização da memória flash com assinatura de código. Fonte: Elaborado pelo autor.

Para gerar um certificado em um cenário de testes, o ESP-IDF proporciona um comando para criar o arquivo e utilizar no projeto. O comando deve ser inserido no terminal do ESP-IDF. Com o certificado gerado, basta informar o nome do arquivo no campo "Secure boot private signing key" do menu de configurações.

```
idf.py secure-generate-signing-key secure_boot_v1_key
```

Com todas as modificações realizadas, um processo de envenenamento de firmware foi realizado intencionalmente no servidor que contém o arquivo a ser utilizado pela atualização OTA. O firmware utilizado para o teste foi um simples exemplo para piscar LEDs sem a assinatura com o certificado gerado, disponível em ([ESPRESSIF, 2025](#)). Abaixo, a saída do dispositivo durante o processo de atualização e exposta e demonstra que não foi possível realizar a atualização OTA devido a falha de autenticação entre o dispositivo e o arquivo de atualização.

```
I (2337) main_task: Returned from app_main()
I (2617) esp-x509-crt-bundle: Certificate validated
I (3277) OTA_JSON: Latest version: 2.0
I (3277) OTA_JSON: Device version: 1.5
I (3277) OTA_JSON: New version found! Starting OTA...
I (4597) esp-x509-crt-bundle: Certificate validated
I (6177) esp-x509-crt-bundle: Certificate validated
I (6967) esp_https_ota: Starting OTA...
I (6967) esp_https_ota: Writing to <ota_0> partition at offset 0x120000
I (30117) esp_image: segment 0: paddr=00120020 vaddr=3f400020 size=19318h (103192) map
I (30157) esp_image: segment 1: paddr=00139340 vaddr=3ffb0000 size=0001ch ( 28)
I (30157) esp_image: segment 2: paddr=00139364 vaddr=3ffb0000 size=03ef0h ( 16112)
I (30167) esp_image: segment 3: paddr=0013d25c vaddr=40080000 size=02dbch ( 11708)
I (30177) esp_image: segment 4: paddr=00140020 vaddr=400d0020 size=83e94h (540308) map
I (30357) esp_image: segment 5: paddr=001c3ebc vaddr=40082dbc size=149dch ( 84444)
I (30387) esp_image: Verifying image signature...
E (30397) secure_boot_v1: image has invalid signature version field 0xffffffff (image without a signature?)
E (30397) esp_image: Secure boot signature verification failed
I (30397) esp_image: Calculating simple hash to check for corruption...
W (30617) esp_image: image valid, signature bad
E (30627) OTA_JSON: OTA failed: ESP_ERR_OTA_VALIDATE_FAILED
```

Figura 12 – Atualização OTA sem assinatura de código. Fonte: Elaborado pelo autor.

3.3.3 Recursos para um Cenário de Produção

O ESP32 oferece recursos avançados de segurança para proteção e segurança do dispositivo. O armazenamento da chave e a checagem da assinatura de código podem ser realizadas por mecanismos de hardware através da funcionalidade de inicialização segura. No exemplo de assinatura de código fornecido no trabalho, apenas a assinatura do firmware e checada. Com a inicialização segura, o bootloader também é validado, o que faz com que até mesmo a atualização do dispositivo via SPI e ESP-IDF sejam validadas. Além disso, o dispositivo possui o recurso para criptografar o firmware e chaves armazenadas no dispositivo, criando uma barreira contra casos de engenharia reversa do firmware e obtenção do certificado de autenticação. Em um cenário de produção, esses recursos são amplamente recomendados (GUANHONG, 2024).

Os eFuses são fusíveis microscópicos programáveis que podem ser "queimados" para armazenar dados no ESP32. Os bits do eFuse são organizados em diferentes campos de dados, que podem ser utilizados tanto para parâmetros do sistema (usados pelo ESP-IDF do ESP32) quanto para parâmetros definidos pelo usuário. O componente eFuse Manager é uma coleção de ferramentas e APIs que auxiliam na definição, gravação e acesso aos parâmetros do eFuse (ESPRESSIF, 2023).

Apesar dos benefícios, a implementação da inicialização segura e da criptografia de flash não foi realizada neste projeto devido à irreversibilidade do processo. A ativação dessas funcionalidades requer a gravação permanente de bits nos eFuses do chip.

3.4 Abordagens e Desempenho

Para capturar questões relacionadas a utilização de recursos do dispositivo ESP32, foram utilizados os dados gerados pelo ESP-IDF no processo de build e gravação, assim como as funções abaixo fornecidas pelo FreeRTOS.

```
uxTaskGetStackHighWaterMark()  
vTaskGetRunTimeStats()
```

A função "uxTaskGetStackHighWaterMark()" permite identificar quantas words, isto é, 4 bytes estão sendo utilizadas na pilha de execução da tarefa na RAM do dispositivo. Já a função "vTaskGetRunTimeStats()" gera um relatório de utilização dos ciclos da CPU para as tarefas sendo executadas no dispositivo.

3.4.1 Desempenho

Os dados coletados durante a execução do firmware de atualização OTA são expostos nesta seção e analisados na [seção 4.1](#).

Conforme mencionado anteriormente, a implementação de assinatura de código impacta no tamanho do bootloader. A seguir, os dados representam que a abordagem base sem assinatura de código tem o bootloader reduzido, com o tamanho de 26,608 bytes. Já a abordagem com assinatura de código apresentou 31,456 bytes de tamanho. Um aumento de aproximadamente 18%. Abaixo, a saída do ESP-IDF durante o processo de build do bootloader.

```
Bootloader binary size 0x67f0 bytes  
Bootloader binary size 0x7ae0 bytes
```

As [Figura 13](#) e [Figura 14](#) apresentam os dados em tempo de execução do uso de CPU das tarefas sendo executadas pelo dispositivo alternando o tempo de busca por um novo firmware. A [Figura 13](#) apresenta uma periodicidade de 5 segundos para um total de 60 segundos. Já para a [Figura 14](#), a periodicidade foi alterada para 60 segundos, mantendo o total.

Task	CPU Ticks	Uso
monitor_task	340,204	<1%
IDLE1	51,675,632	91%
IDLE0	54,410,057	95%
tiT	232,857	<1%
ota_update	5,393,854	9%
blink_task	23,774	<1%
ipc0	7,744	<1%
sys_evt	15,996	<1%
esp_timer	89,767	<1%
wifi	1,245,470	2%
Tmr Svc	21	<1%
ipc1	33,764	<1%

Stack High Watermark: 5292 words | Iteração: 12

Figura 13 – Tarefa ota_update com periodicidade de 5 segundos. Fonte: Elaborado pelo autor.

Task	CPU Ticks	Uso
monitor_task	333,003	<1%
IDLE1	56,007,566	98%
IDLE0	55,035,521	97%
tiT	63,615	<1%
ota_update	537,279	<1%
blink_task	20,301	<1%
ipc0	7,728	<1%
Tmr Svc	21	<1%
esp_timer	87,345	<1%
wifi	1,038,144	1%
ipc1	33,753	<1%
sys_evt	4,471	<1%

Stack High Watermark: 5292 words | Iteração: 12

Figura 14 – Tarefa ota_update com periodicidade de 60 segundos. Fonte: Elaborado pelo autor.

3.4.2 Abordagem Automatizada

Para realizar o processo de atualização FOTA, existem duas possíveis abordagens que podem ser exploradas e afetam a segurança e o desempenho do dispositivo realizando o processo de atualização.

A primeira abordagem consiste em um método que realiza checagens periódicas para determinar se deve ou não realizar o processo de atualização. Na implementação desenvolvida neste trabalho, o processo de atualização busca por um novo firmware de acordo com um período definido para a tarefa de busca. O período definido para a varredura influencia a utilização de recursos do dispositivo.

A interação da função de checagem por atualização OTA com o dispositivo pode ser resumida nas duas etapas abaixo.

- Utilização de recursos de rede para criar a conexão e acessar o arquivo JSON;

- Utilização de recursos de CPU e memória para processar o arquivo JSON;

A utilização dos recursos do dispositivo se torna baixa de acordo com o aumento do intervalo de tempo de varredura, portanto essa abordagem foi escolhida para ser implementada no trabalho. Os detalhes de utilização de recursos serão abordados na [seção 4.1](#).

3.4.3 Abordagem Sob Demanda

Além da abordagem automatizada, o processo de atualização de firmware do dispositivo pode ser realizado sob demanda, isto é, por um comando ao dispositivo ou evento. Desta maneira, existe a possibilidade de efetuar a varredura utilizando recursos de CPU, porém sem a necessidade de processar um arquivo JSON. Outra abordagem que pode ser explorada é a utilização de interrupções para captar um determinado comando que aciona a função responsável por realizar a atualização OTA. O trade-off entre recursos de CPU, RAM e interrupções depende da aplicação em questão.

4 Análises e Resultados

4.1 Análise de Desempenho

Como citado por ([ARAKADAKIS et al., 2021](#)), os dispositivos IoT geralmente são equipados com recursos escassos, como memória, armazenamento e poder de processamento limitados, a fim de manter baixo o custo e o consumo de bateria. Essas características impõem desafios adicionais tanto no projeto do firmware que os nós executam quanto no processo de atualização ao longo do tempo.

A compreensão da interação entre as funcionalidades de segurança e o processo de atualização OTA com os recursos disponíveis é essencial para orientar de forma estratégica futuras implementações. Essa análise permite identificar eventuais limitações e apresentar um panorama de como alterações no fluxo de atualização OTA refletem no desempenho do dispositivo.

4.1.1 Tempo de CPU

Os dados das [Figura 13](#) e [Figura 14](#) representam o quanto tempo o processador do ESP32 passou executando uma tarefa. As tarefas se alternam entre os estados executando, pronta, bloqueada, suspensa e excluída como mencionado anteriormente na [subseção 2.9.1](#). Neste caso, a métrica se refere a quantidade de ciclos de CPU com a tarefa no estado Executando.

De acordo com os dados obtidos, a tarefa de realizar a busca por um novo firmware utiliza <1% dos ciclos da CPU para realizar a cada 60 segundos. Já a aumentando o tempo de busca para 5 segundos, a utilização da CPU passa a ser de 9%. A abordagem automatizada de busca por um novo firmware OTA apresenta baixo uso de CPU. Se o dispositivo for configurado para buscar por uma atualização 1 vez ao dia, a utilização de recursos se torna consideravelmente baixa. Por esse motivo, a abordagem automatizada é considerada mais abrangente para diferentes tipos de aplicações em relação a abordagem sob demanda.

4.1.2 Memória Flash

Conforme os dados de tamanho do bootloader na [subseção 3.4.1](#), é possível identificar o aumento de utilização da memória flash por parte do bootloader de acordo com a adição de mecanismos de segurança como a assinatura de código. Isto ocorre porque as funções responsáveis por realizar o processo de verificação e armazenamento do certificado precisam ser incorporados ao bootloader. O aumento de aproximadamente 18% na utilização de memória flash para a realização da verificação da assinatura de código pode ser um impeditivo para cenários com escassez de memória flash. A alternativa apresentada na [subseção 3.3.3](#) é considerada

mais segura segundo (ESPRESSIF, 2023), porém existe a dependência de hardware dedicado. Os circuitos dedicados realizam a operação de inicialização segura e armazena o certificado em memórias que podem ser programadas uma única vez. Com tal recurso, a adulteração do dispositivo se torna consideravelmente restrita, uma vez que o certificado não pode ser alterado e a funcionalidade de assinatura do código será realizada independentemente do bootloader. Caso o bootloader seja corrompido, o processo de verificação ainda ocorrerá por parte do dispositivo dedicado.

4.2 Análise de Segurança

Os ataques MITM permitem que um ator intercepte, visualize e altere dados confidenciais. MITM é um tipo genérico de ataque em que um invasor se insere entre a vítima e a parte pretendida durante a comunicação. Os invasores podem lançar ataques MITM para aumentar a exploração, roubar informações, anular sistemas de autenticação e fazer-se passar por vítimas ou agir como vítimas (GRAHAM; OLSON; HOWARD, 2016).

Nas implementações e abordagens analisadas nesta pesquisa, os riscos relacionados a ataques do tipo MITM e à adulteração de firmware foram analisados e simulados para obter o resultado de um possível caso de ataque real seguindo as implementações propostas.

A Figura 15 demonstra como a tríade de cibersegurança é comprometida através das ameaças que afetam a transmissão e recepção/installação do firmware OTA.

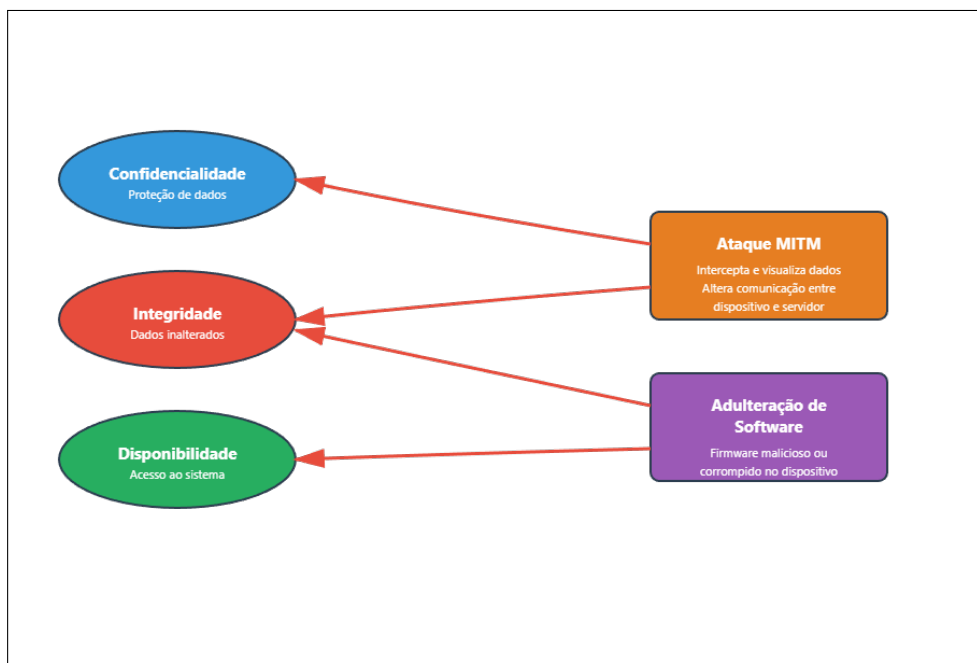


Figura 15 – Tríade de cibersegurança e ameaças. Fonte: Elaborado pelo autor.

4.2.1 Implementação Utilizando HTTP/HTTPS

Durante os testes de atualização OTA realizado no dispositivo ESP32, foi utilizada uma abordagem comparativa entre os protocolos HTTP e HTTPS. Inicialmente, o processo foi executado utilizando HTTP, onde o arquivo JSON com as informações do binário de firmware foram acessados de forma não criptografada e sem a autenticação com o servidor hospedando os arquivos JSON e o firmware. Ao simular a substituição das informações contidas no JSON por versões alteradas propositalmente, o dispositivo concluiu a atualização sem restrições, indicando ausência de verificação de integridade e expondo o sistema a possíveis comprometimentos.

Na segunda etapa, o protocolo HTTPS foi adotado como alternativa para garantir a confidencialidade e autenticidade dos dados em trânsito. Os testes mostraram que, mesmo com tentativas de alteração do dados no JSON, o ESP32 não realizou o download do firmware adulterado. Contudo, observou-se que o comprometimento do servidor HTTPS ainda representa um vetor de ataque relevante, pois permite a entrega de firmware malicioso mesmo sob conexão segura, o que indica limitações no uso exclusivo da segurança de transporte para proteger a autenticidade do conteúdo. Conforme apresentado por (BIELAWSKI et al., 2020), as etapas do processo de atualização OTA apresentam diferentes riscos e diferentes tipos de mitigação.

4.2.2 Implementação Utilizando Assinatura de Código

Dando continuidade à análise anterior, foi incorporado o mecanismo de assinatura de código ao processo OTA como medida complementar de segurança. A assinatura digital do binário permite que o dispositivo valide criptograficamente a autenticidade e integridade do firmware antes de aplicar a atualização. Nos testes realizados, o ESP32 rejeitou firmwares alterados ou não assinados, mesmo quando entregues por servidores comprometidos, conforme apresentado na [subseção 3.3.2](#)

Esse resultado evidencia que a assinatura de código atua como um filtro de confiança, bloqueando atualizações não autorizadas e assegurando que apenas firmware proveniente de fontes legítimas seja executado. Em conjunto com o uso do protocolo HTTPS, essa abordagem estabelece um duplo mecanismo de proteção que mitiga significativamente os riscos de comprometimento. A integração dessas duas camadas — segurança na comunicação e verificação de autenticidade do firmware — é fundamental para preservar a integridade dos dispositivos embarcados em ambientes vulneráveis, conforme abordado por (BIELAWSKI et al., 2020) e (HAMADA, 2020).

Por fim, a implementação FOTA tem o seu fluxo de execução exposta na [Figura 16](#).

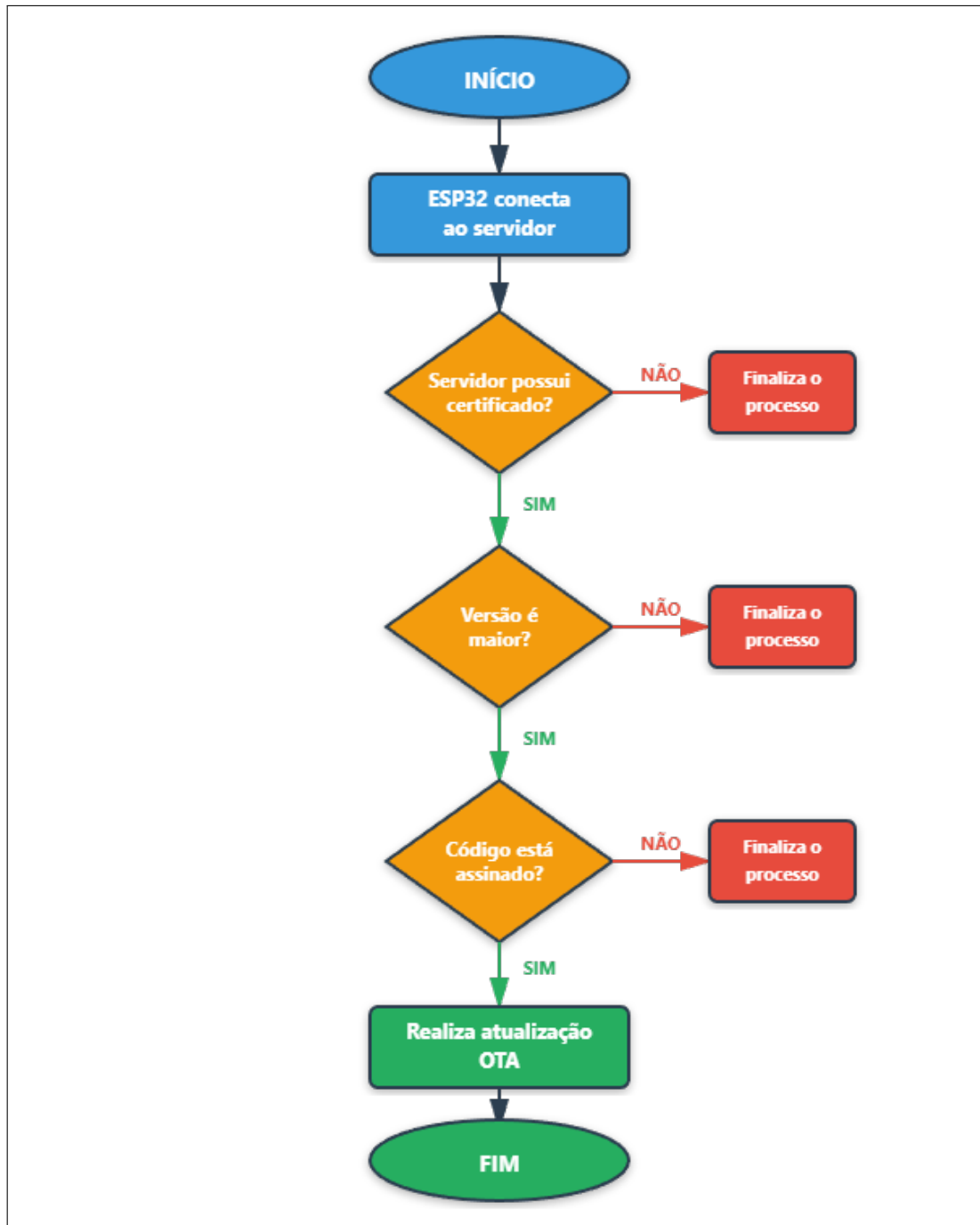


Figura 16 – Fluxo de execução FOTA. Fonte: Elaborado pelo autor.

4.2.3 Rollback

O rollback de aplicação no ESP32 é um mecanismo de segurança projetado para manter o dispositivo operacional após atualizações OTA, permitindo o retorno automático à versão anterior quando a nova aplicação apresenta problemas críticos. Este sistema funciona através de estados que o bootloader utiliza para determinar qual versão da aplicação deve ser carregada durante a inicialização. A flag responsável por habilitar o rollback de firmware é listada abaixo.

`CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE`

Quando uma atualização OTA é instalada, a aplicação pode seguir três caminhos distintos: se funcionar corretamente, ela chama `esp_ota_mark_app_valid_cancel_rollback()` para marcar-se com estado `ESP_OTA_IMG_VALID`, confirmando sua estabilidade; se detectar erros críticos que impedem seu funcionamento, executa `esp_ota_mark_app_invalid_rollback_and_reboot()` para marcar-se como `ESP_OTA_IMG_INVALID` e reiniciar, forçando o bootloader a carregar a versão anterior; ou, caso a opção `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` esteja habilitada e ocorra um reset sem que nenhuma função de validação tenha sido chamada, o rollback acontece automaticamente como medida preventiva. O fluxo recomendado envolve realizar diagnósticos na primeira inicialização após a atualização, validando explicitamente a aplicação em caso de sucesso ou acionando o rollback manualmente em caso de falha. Para situações onde a aplicação não consegue nem executar devido a travamentos, abortos ou perdas de energia, o próprio bootloader detecta a anomalia na próxima tentativa de boot e realiza o rollback automaticamente, marcando a versão problemática como inválida e restaurando a aplicação anterior que estava funcionando, garantindo assim a recuperação do dispositivo sem intervenção manual (ESPRESSIF, 2023).

O desenvolvedor tem flexibilidade para implementar sua própria lógica de verificação, podendo executar diagnósticos personalizados baseados em funcionalidades específicas da aplicação ou realizar a validação após a inicialização completa do sistema. Esta verificação pode incluir testes de conectividade, validação de sensores, checagem de periféricos ou qualquer outro critério relevante para determinar se a nova versão está operando adequadamente antes de confirmar sua validade através da chamada da função de validação.

4.2.4 Vulnerabilidades

Funcionalidades escondidas em software e hardware é um tópico bastante discutido na computação em geral. Embora o recurso oculto não seja diretamente malicioso, aumenta a superfície de exposição do produto para ataques maliciosos. Tal fraqueza já é enumerada desde 2012 como CWE-912 (CORPORATION, 2012). Como exemplo da fraqueza citada e de uma exploração a partir disso, é interessante citar o caso de um dispositivo de monitoramento de pacientes utilizado nos Estados Unidos pelo U.S. Healthcare and Public Health (HPH). O firmware do dispositivo apresentava um endereço IP explícito nas linhas de código, o que gerou um backdoor de acesso pela internet para o dispositivo, comprometendo os dados dos pacientes (CISA, 2025). A vulnerabilidade foi, posteriormente, enumerada também como CVE-2025-0626 (CORPORATION, 2025b).

Em março de 2025, foi descoberta e registrada uma grave falha de segurança nos dispositivos Espressif ESP32, amplamente utilizados em aplicações de Internet das Coisas (IoT) devido à sua alta versatilidade e suporte a diversos protocolos de comunicação. A exploração dessa falha

resultou na criação de uma vulnerabilidade enumerada que foi formalmente registrada em bases de dados de segurança como a CVE. A vulnerabilidade em questão está diretamente relacionada aos comandos HCI (Host Controller Interface), que representam uma lista padronizada de instruções no protocolo Bluetooth. Esses comandos são responsáveis por facilitar a comunicação entre o host (software) e o controlador Bluetooth (hardware), permitindo a troca de dados e o gerenciamento de funções do dispositivo Bluetooth. Os comandos HCI desempenham um papel essencial na pilha do Bluetooth, pois servem como intermediários entre o sistema operacional ou firmware e o módulo de comunicação Bluetooth.

A [Tabela 2](#) apresenta as 29 instruções não documentadas, obtidas através de ([TARLOGIC, 2025](#)).

Opcode	Command
OxFC01	Read memory
OxFC02	Write memory
OxFC03	Delete NVDS parameter
OxFC05	Get flash ID
OxFC06	Erase flash
OxFC07	Write flash
OxFC08	Read flash
OxFC09	Read NVDS parameter
OxFC0A	Write NVDS parameter
OxFC0B	Enable/disable coexistence
OxFC0E	Send LMP packet
OxFC10	Read kernel stats
OxFC11	Platform reset
OxFC12	Read memory info
OxFC30	Register read
OxFC31	Register write
OxFC32	Set MAC address
OxFC35	Set CRC initial value
OxFC36	LLCP msgs discard
OxFC37	Reset RX count
OxFC38	Reset TX count
OxFC39	RF register read (Não implementado)
OxFC3A	RF register write (Not implementado)
OxFC3B	Set TX password
OxFC40	Set LE parameters
OxFC41	Write LE default values
OxFC42	LLCP pass through enable
OxFC43	Send LLCP packet
OxFC44	LMP msgs discard

Tabela 2 – Instruções não documentadas do ESP32. Fonte: ([TARLOGIC, 2025](#)).

Funcionalidades escondidas em dispositivos embarcados podem oferecer grandes riscos.

Engenheiros e desenvolvedores de dispositivos que forneçam a tecnologia FOTA devem se atentar para não aumentar a exposição a ataques do dispositivo em questão, visto que se um ataque realizar a adulteração de um dispositivo fisicamente, todos os outros podem ser comprometidos remotamente.

5 Conclusão

A presente pesquisa abordou os principais aspectos relacionados à FOTA em sistemas embarcados, destacando os riscos e desafios associados à sua implementação. Com base na análise de referências técnicas, padrões de segurança e experimentações realizadas, foi possível identificar um conjunto de requisitos fundamentais para garantir a segurança e a confiabilidade do processo de atualização.

Dentre esses requisitos, destacam-se: o uso de um bootloader em dois estágios, pontuados como um trade-off necessário por (BROWN, 2018) e (TINDELL, 2023), o que permite maior controle e verificação durante a inicialização do dispositivo. A Figura 17 demonstra as 3 variações mais comuns de implementação de bootloader. Bootloader unificado ao firmware, bootloader fixo e separado em memória ROM e bootloader dividido em dois estágios na ROM e FLASH. A abordagem mais à direita apresenta maior flexibilidade e robustez, conforme discutido na pesquisa.

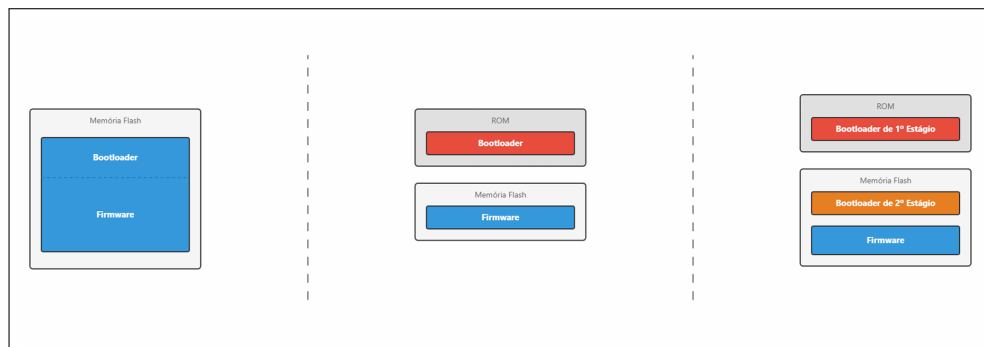


Figura 17 – Abordagens de bootloader. Fonte: Elaborado pelo autor.

A utilização do protocolo HTTPS para garantir a autenticidade e a confidencialidade na transmissão dos pacotes de firmware. Também foi demonstrado que a adoção de partições duplicadas para OTA, possibilita mecanismos de rollback para versões anteriores em caso de falha, o que garante o funcionamento do dispositivo mesmo que o processo OTA falhe. Como ponto principal, foi apresentado que assinatura de código digital assegura a autenticidade do firmware na etapa de instalação, conforme introduzido por (BIELAWSKI et al., 2020) e (HAMADA, 2020).

Adicionalmente, observou-se que, conforme o grau de criticidade da aplicação, é recomendável adotar práticas avançadas de segurança, como a inicialização segura e a criptografia de flash, seguindo o modelo do ESP32, de forma a dificultar significativamente ataques por engenharia reversa e impossibilitar que, mesmo que o bootloader seja adulterado, a verificação de assinatura de código se mantenha eficaz, abordado em (ESPRESSIF, 2023).

Como contribuição prática, a pesquisa também oferece um guia replicável, permitindo que desenvolvedores e estudantes implementem e explorem soluções OTA de forma segura utilizando o ESP32, um dispositivo de desenvolvimento acessível. Espera-se, com isso, promover a disseminação do conhecimento sobre essa tecnologia essencial no contexto de FOTA e elencar praticas essenciais para desenvolver a tecnologia.

Referências

AL-SARAWI, S. et al. Internet of things (iot) communication protocols. In: IEEE. *2017 8th International conference on information technology (ICIT)*. [S.l.], 2017. p. 685–690. Citado na página 33.

ALMEIDA, R. M. A. de; MORAES, C. H. V. de; SERAPHIM, T. d. F. P. *Programação de sistemas embarcados: Desenvolvendo software para microcontroladores em linguagem C*. [S.l.]: Elsevier Brasil, 2017. Citado 2 vezes nas páginas 22 e 28.

ANALYTICS, I. *IoT Market Size Reached \$269 Billion in 2023, with Growth Deceleration in 2024*. 2024. Accessed: 2025-05-25. Disponível em: <<https://iot-analytics.com/iot-market-size/>>. Citado 3 vezes nas páginas 7, 16 e 17.

ARAKADAKIS, K. et al. Firmware over-the-air programming techniques for iot networks-a survey. *ACM Computing Surveys (CSUR)*, ACM New York, NY, v. 54, n. 9, p. 1–36, 2021. Citado 4 vezes nas páginas 25, 31, 32 e 61.

BARRY, R. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016. Acessado em: 14 de julho de 2025. Disponível em: <https://www.freertos.org/media/2018/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf>. Citado na página 35.

BENINGO, J. Bootloader design for microcontrollers in embedded systems. *Embedded Software Design Techniques*, 2015. Citado na página 32.

BIELAWSKI, R. et al. *Cybersecurity of firmware updates*. [S.l.], 2020. Citado 6 vezes nas páginas 23, 24, 26, 27, 63 e 69.

BILEK, J.; RUZICKA, I. Evolutionary trends of embedded systems. In: *IEEE International Conference on Industrial Technology, 2003*. [S.l.: s.n.], 2003. v. 2, p. 901–905 Vol.2. Citado 2 vezes nas páginas 19 e 29.

BROWN, B. B. Over-the-air (ota) updates in embedded microcontroller applications: Design trade-offs and lessons learned. *Analog Dialogue Technical Journal*, v. 52, p. 52–11, 2018. Citado 2 vezes nas páginas 23 e 69.

CISA. *ESP32 hidden HCI vendor commands, technical details and use cases*. 2025. <<https://www.cisa.gov/news-events/alerts/2025/01/30/cisa-releases-fact-sheet-detailing-embedded-backdoor-function-contec-cms8000-firmware>>. Accessed: March 26, 2025. Citado 2 vezes nas páginas 27 e 65.

CORPORATION, M. *Common Weakness Enumeration*. 2012. <<https://cwe.mitre.org/data/definitions/912.html>>. Accessed: March 24, 2025. Citado 2 vezes nas páginas 27 e 65.

CORPORATION, M. *Common Vulnerabilities and Exposures*. 2025. <<https://www.cve.org/CVERecord?id=CVE-2025-27840>>. Accessed: March 24, 2025. Citado na página 27.

CORPORATION, M. *CVE-2025-0626: Hidden Functionality Vulnerability in Contec CMS8000 Patient Monitor*. 2025. Accessed: 2025-03-26. Disponível em: <<https://www.cve.org/CVERecord?id=CVE-2025-0626>>. Citado na página 65.

ESPRESSIF. *ESP-IDF Programming Guide*. 2023. Disponível em: <<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/index.html>>. Acesso em: 08 de agosto de 2023. Citado 9 vezes nas páginas 7, 29, 35, 40, 43, 56, 62, 65 e 69.

ESPRESSIF. *ESP-IDF: Espressif IoT Development Framework*. 2025. <<https://github.com/espressif/esp-idf>>. Repositório oficial no GitHub. Citado 2 vezes nas páginas 44 e 55.

FREERTOS. *Documentação do Kernel FreeRTOS*. 2025. Acessado em: 14 de abril de 2025. Disponível em: <<https://www.freertos.org/Documentation/00-Overview>>. Citado na página 36.

FREERTOS. *Task States - FreeRTOS*. 2025. Accessed: 2025-05-29. Disponível em: <<https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/02-Task-states>>. Citado 2 vezes nas páginas 7 e 36.

GRAHAM, J.; OLSON, R.; HOWARD, R. *Cyber security essentials*. [S.l.]: CRC Press, 2016. Citado na página 62.

GUANHONG, C. *Utilizando a Ferramenta de Gravação Flash da ESP para habilitar Criptografia de Flash, Boot Seguro e Criptografia NVS*. 2024. Espressif Developer Portal. Disponível em: <<https://developer.espressif.com/blog/flash-encryption-and-secure-boot/>>. Citado na página 56.

HAMADA, K. *Firmware over the air*. Tese (Doutorado) — Faculty of Engineering at Cairo University in Partial Fulfilment of the . . . , 2020. Citado 6 vezes nas páginas 7, 15, 16, 22, 63 e 69.

HENNESSY, J. L.; PATTERSON, D. A. *Computer organization and design RISC-V edition: The hardware software interface*. [S.l.]: Elsevier Science & Technology Books, 2017. Citado na página 19.

KAMAL, M. A. et al. Requirements, deployments, and challenges of lora technology: A survey. *Computational Intelligence and Neuroscience*, Wiley Online Library, v. 2023, n. 1, p. 5183062, 2023. Citado na página 34.

LEA, P. *Internet of Things for Architects: Architecting IoT solutions by implementing sensors, communication infrastructure, edge computing, analytics, and security*. [S.l.]: Packt Publishing Ltd, 2018. Citado 3 vezes nas páginas 33, 34 e 35.

LEE, E. A.; SESHIA, S. A. *Introduction to embedded systems: A cyber-physical systems approach*. [S.l.]: Mit Press, 2016. Citado 3 vezes nas páginas 20, 22 e 32.

LI, B. et al. Over-the-air upgrading for enhancing security of intelligent connected vehicles: a survey. *Artificial Intelligence Review*, Springer, v. 57, n. 11, p. 314, 2024. Citado na página 26.

MEHAR, M. et al. Over the air (ota) update system—a systematic. Citado 2 vezes nas páginas 15 e 27.

NEUSTEIN, A. et al. *AI, IoT, Big Data and Cloud Computing for Industry 4.0*. [S.l.]: Springer, 2023. Citado 2 vezes nas páginas 13 e 15.

PAKPAHAN, A. C. et al. Comparative study between harvard and von neumann architectures in modern computing applications. Citado na página 21.

PARK, C.-Y.; LEE, S.-J.; LEE, I.-G. Secure and lightweight firmware over-the-air update mechanism for internet of things. *Electronics*, v. 14, n. 8, 2025. ISSN 2079-9292. Disponível em: <<https://www.mdpi.com/2079-9292/14/8/1583>>. Citado na página 25.

PULE, M.; ABU-MAHFOUZ, A. M. Firmware updates over the air mechanisms for low power wide area networks: A review. In: IEEE. *2019 International Multidisciplinary Information Technology and Engineering Conference (IMITEC)*. [S.l.], 2019. p. 1–7. Citado 4 vezes nas páginas 13, 17, 30 e 31.

QUADRI, A.; SIDEK, B. O. An introduction to over-the-air programming in wireless sensor networks. *Int J Comput Sci Netw Solut [Internet]*, Citeseer, v. 2, p. 33–49, 2014. Citado na página 30.

RAHMAN, F. et al. Hardware-assisted cybersecurity for iot devices. In: IEEE. *2017 18th International Workshop on Microprocessor and SOC Test and Verification (MTV)*. [S.l.], 2017. p. 51–56. Citado na página 22.

SEN, J. A survey on wireless sensor network security. *arXiv preprint arXiv:1011.1529*, 2010. Citado 3 vezes nas páginas 13, 27 e 31.

STENBERG, E. *Digital health, medical devices and OTA software updates*. 2021. Disponível em: <<https://www.devicechronicle.com/digital-health/>>. Acesso em: 30 de maio de 2023. Citado na página 26.

SYSTEMS, C. *Cisco Annual Internet Report*. 2020. Último acesso em 20 de maio de 2025. Disponível em: <<https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>>. Citado na página 15.

TANENBAUM, A. S. *Structured computer organization*. [S.l.]: Pearson Education India, 2016. Citado 4 vezes nas páginas 19, 20, 21 e 29.

TARLOGIC. *ESP32 hidden HCI vendor commands, technical details and use cases*. 2025. <<https://www.tarlogic.com/blog/esp32-hidden-hci-vendor-commands/>>. Accessed: March 24, 2025. Citado 3 vezes nas páginas 8, 27 e 66.

TENG, C.-C. et al. Firmware over the air for home cybersecurity in the internet of things. In: IEEE. *2017 19th Asia-Pacific Network Operations and Management Symposium (APNOMS)*. [S.l.], 2017. p. 123–128. Citado na página 30.

TINDELL, D. K. *Digital health, medical devices and OTA software updates*. 2023. <<https://kentindell.github.io/2023/04/18/get-your-app-to-mars/>>. Disponível em: <<https://kentindell.github.io/2023/04/18/get-your-app-to-mars/>>. Acesso em: 15 de maio de 2023. Citado 4 vezes nas páginas 13, 23, 32 e 69.

Apêndices

APÊNDICE A – Informações Adicionais

Todos os códigos utilizados no documento estão disponíveis no [GitHub](#) do autor.